Modeling and Simulation — Lesson 1

The Limits to Growth: Basic Population Models

Václav B. Petrák

Faculty of Biomedical Engineering
Czech Technical University in Prague

February 18, 2026

Content

Introduction

Contacts

Lecturer: Václav B. Petrák
Email: vaclav.petrak@fbmi.cvut.cz
Phone/WhatsApp: +420 725 878 390

Consultations:

- In person: By individual request. Please email or text me.
- In person: Friday, May 2nd, 2025 (Project-related problems).
- Via email: Anytime.

Absence Policy

- **Advance Notice Required:** All absences must be excused in advance whenever possible.
- **Communication Procedure:** Contact me as soon as possible by email with the date of absence and the reason.
- **Responsibility for Missed Work:** You are responsible for making up missed assignments and catching up on course materials.

Lectures

- We will cover 5 topics in modeling and simulation, one per session.
- The last lecture will be focused on student presentations.

Classification

There is no exam.

Your grade will be based on your independent work.
There are three outputs used for grading:

1. **Journal Club Presentation**
   30% of the final grade

2. **Capstone Project Presentation**
   40% of the final grade

3. **Code Review Discussion**
   30% of the final grade

Journal Club is a discussion of research papers

A journal club is a group that meets to critically evaluate articles in academic literature. Journal clubs:

- Encourage critical thinking.
- Help in understanding the studied concepts and methods.
- Help keep up with the latest research.
- Improve communication and presentation skills.
- Simulate the experience of a scientist.

The main purpose in this course is to explore how the methods we learned are applied in current research.

Your Task for Journal Club

- You will read a research paper related to the topic of your capstone project.
- Your task is to read the paper and prepare a 15-minute presentation, followed by a 5-minute discussion, to explain the paper.
- The presentation dates will be arranged individually.

The Journal Club presentation will cover:

Introduction: Provide a high-level overview of the background of the research topic and the main objectives of the paper.

Methods: Give an overview of the methodology used in the paper.

Results: Highlight the key results and conclusions.

Discussion: Discuss the practical implications of the research, its applications in real-world situations, and any limitations.

Conclusions: Summarize the main contributions and overall findings of **the paper**. Reiterate its key takeaways.

Q&A: Answer questions from the audience (lecturer).

Grading Criteria for Journal Club Presentation

Presentation Design and Delivery (50%)

- **Design:** The presentation is well-organized and easy to follow.
- **Timing:** The presenter adhered to the 15-minute format.
- **Delivery:** Clear and confident speech with appropriate pacing.
- **Questions:** Convincing response to questions.

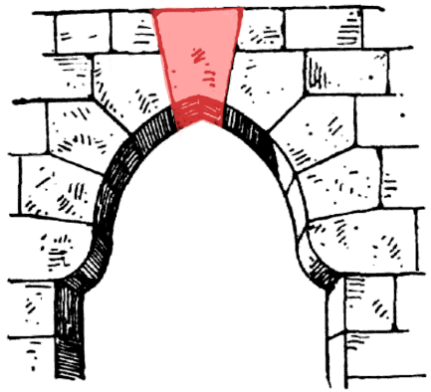
Grading Criteria for Journal Club Presentation

Content of the Presentation (50%)

- **Introduction:** Provides a high-level overview, explaining the motivation and research question.
- **Methods:** Explains the experimental approach used by the authors. Unfamiliar methods are explained without excessive detail.
- **Results:** The key results from the paper are explained.
- **Critical Evaluation:** Critique of methods, ideas for improvement, and significance of the results.

Capstone project is your independent project

- You will work on an independent project that builds on and extends course content.
- The output of your work will be a presentation and the code.
- Presentation dates will be arranged individually.



*A **capstone** (or keystone) is the wedge-shaped stone at the apex of an arch. **It is the final piece placed during construction and locks all the stones into position**, allowing the arch to bear weight.*

The Capstone Project presentation will cover:

Introduction: Provide a high-level overview of the background and main objectives of **your project**.

Methods: Give an overview of the methodology used in **your project**.

Results: Highlight the key results and conclusions.

Discussion: Discuss the practical implications of **your findings** and their applications. Discuss any limitations.

Conclusions: Summarize the main contributions and overall findings. Reiterate key takeaways.

Q&A: Answer questions from the audience (lecturer).

Grading Criteria for Capstone Project

Presentation Design and Delivery (50%)

- **Design:** The presentation is well-organized and easy to follow.
- **Plots:** Clear graphical presentation of simulation results that address project objectives and show model behavior.
- **Timing:** The presenter adhered to the 15-minute format.
- **Delivery:** Clear and confident speech with appropriate pacing.
- **Questions:** Convincing response to questions.

Grading Criteria for Capstone Project

Model Implementation (50%)

- **Correctness:** Correct functionality of fundamental models.
- **Understanding:** Demonstrated understanding of core concepts.
- **Extension:** Successful and meaningful implementation of additional features (if applicable).
- **Discussion:** Insightful discussion on how parameters/initial conditions affect outcomes. Identification and explanation of behaviors.

During Code Review we will discuss your code

Code Review assesses **your understanding**, quality, design, and documentation of the Capstone Project code.

- **Primary Focus:** The review concentrates on the Capstone Project code.
- **Interactive Session:** An interactive code walk-through where you explain your code's functionality and design choices.
- **Demonstrating Ownership:** An opportunity to demonstrate your understanding, justify development decisions, and articulate the purpose of code segments.
- **Goal:** Ensure the correctness of your project while demonstrating your grasp of the implemented concepts.

Grading Criteria for Code Review

Readability and Maintainability (20%)

- Clean, well-formatted, and easy-to-follow code.
- Consistent naming conventions and style.
- Logical organization: functions that break code into manageable components where necessary.
- Sufficient and meaningful comments for complex/non-obvious code.

Grading Criteria for Code Review

Code Understanding (80%)

- Ability to explain how any part of the code works.
- Ability to clearly articulate design decisions and justify coding choices.
- Ability to suggest improvements, extensions, or alternatives.

Avoid using AI to solve entire exercises

- Consider a math problem:
  - *Given that two trains are traveling on parallel tracks in opposite directions with speeds of 56 km/h and 52 km/h... after what duration will they cross each other?*
- The primary goal is **not** to find the answer but to practice your reasoning and calculation skills.
- Similarly, the aim of modeling and simulation exercises is to learn and practice new skills.
- Avoid using generative AI like ChatGPT or Gemini to solve the **entire problem** for you.
  - It spoils the learning process for you.
  - It may disrupt the pacing of the lesson.

Using AI During Lessons

During lessons, you may use AI to:

- Ask clarifying questions.
- Debug your code (after you've made your own attempts).
- Get ideas for further work.

Let's discuss what constitutes fair use of generative AI in the capstone project.

A model is a representation of a **system**

- A model is a representation of a **system** that enables us to investigate its properties and **predict** future outcomes.
  - A **system** is a set of components that interact with each other within a boundary to function as a whole (e.g., Solar system, rabbit and fox populations).
- In modeling, we aim to identify components, relationships, and behaviors to predict system dynamics.

A model is a representation of a **system**

- A model is a representation of a **system** that enables us to investigate its properties and **predict** future outcomes.
  - A **system** is a set of components that interact with each other within a boundary to function as a whole (e.g., Solar system, rabbit and fox populations).
- In modeling, we aim to identify components, relationships, and behaviors to predict system dynamics.
- Modeling always requires simplification (abstraction).

Example of a model: SIR (susceptible, infectious, and recovered) model for dynamics of infectious diseases

A model is a representation of a **system**

- A model is a representation of a **system** that enables us to investigate its properties and **predict** future outcomes.
  - A **system** is a set of components that interact with each other within a boundary to function as a whole (e.g., Solar system, rabbit and fox populations).
- In modeling, we aim to identify components, relationships, and behaviors to predict system dynamics.
- Modeling always requires simplification (abstraction).
- **A mathematical model uses mathematical equations to describe a system.**

SIR model equations:

$$\frac{dS}{dt} = -\beta SI$$
$$\frac{dI}{dt} = \beta SI - \gamma I$$
$$\frac{dR}{dt} = \gamma I$$

All models are approximations. Assumptions, whether implied or clearly stated, are never exactly true.

All models are wrong, but some models are useful.

So the question you need to ask is not *"Is the model true?"*

It never is.

But *"Is the model good enough for this particular application?"*

— George E. P. Box

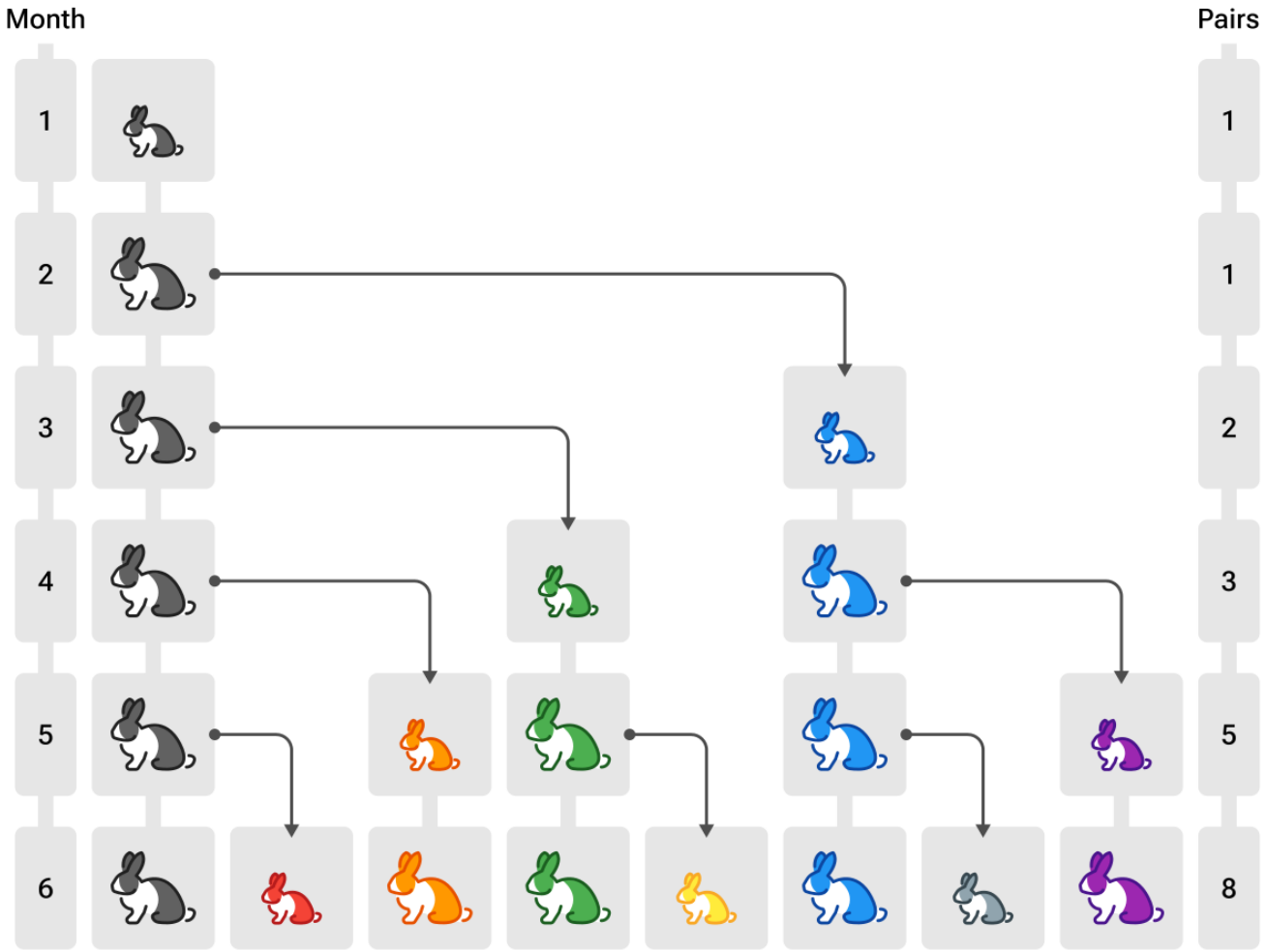
Section 1: Fibonacci Sequence

A very simple population model

The Italian mathematician Leonardo Fibonacci (1170 - 1250) considered the following conditions when calculating the number of rabbits born over the course of one year:

- A single newly born pair of rabbits (one male, one female) is put in a field.
- Rabbits are able to mate at the age of one month, so that at the end of their second month, a female can produce another pair of rabbits.
- Rabbits never die, and a mating pair always produces one new pair (one male, one female) every month from the second month on.

Graphical representation of the first 6 months



Fibonacci sequence

This problem leads to a sequence of numbers called the **Fibonacci sequence**:

$$1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, ...$$

The first two numbers are given, and subsequent numbers are found by adding the two preceding numbers. Thus, we have the following recursive definition (a definition that defines elements in a set in terms of other elements in the set) for Fibonacci numbers:

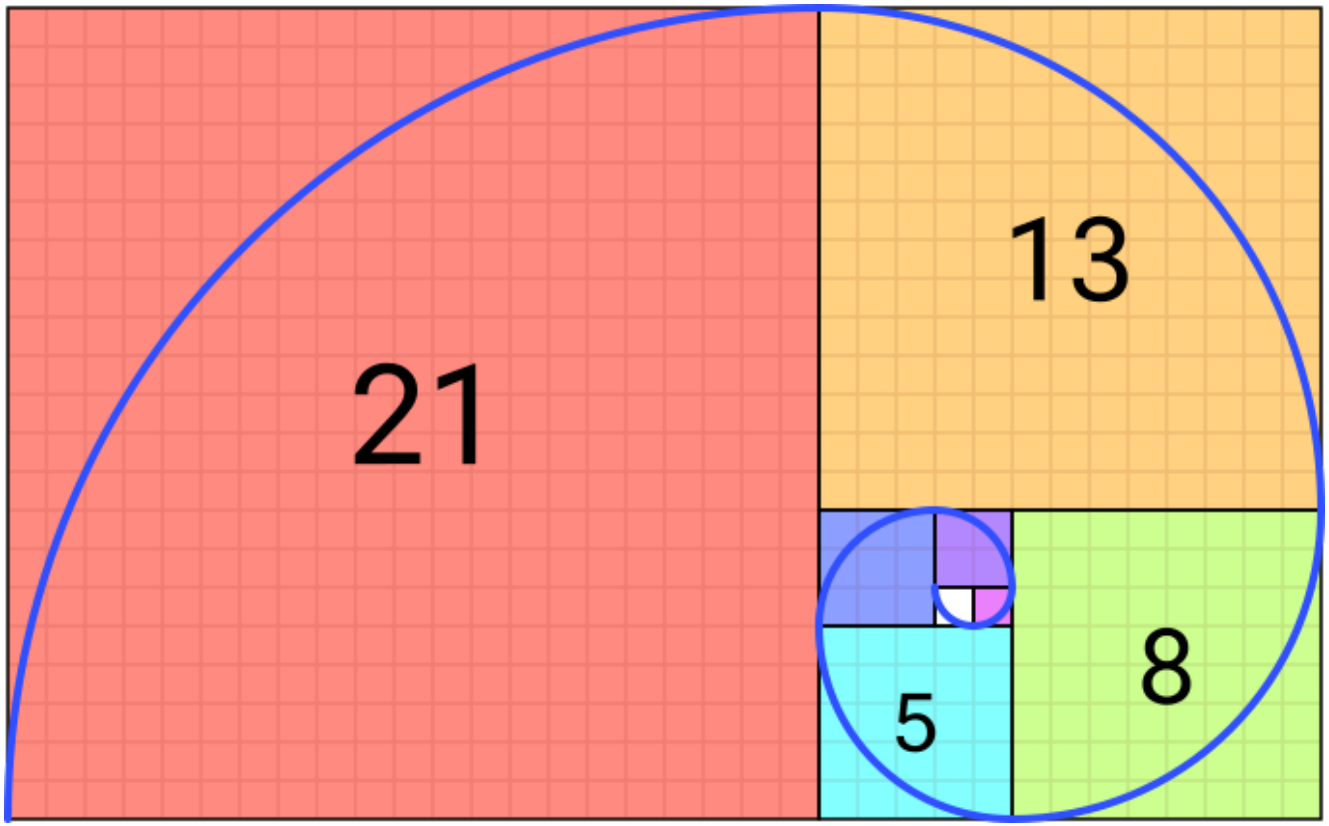
$$F_1 = F_2 = 1$$
$$F_n = F_{n-1} + F_{n-2}$$

The ratio of two consecutive Fibonacci numbers converges to a number known as the **golden ratio**:

$$\psi = \frac{1 + \sqrt{5}}{2} \approx 1.618... \tag{1}$$

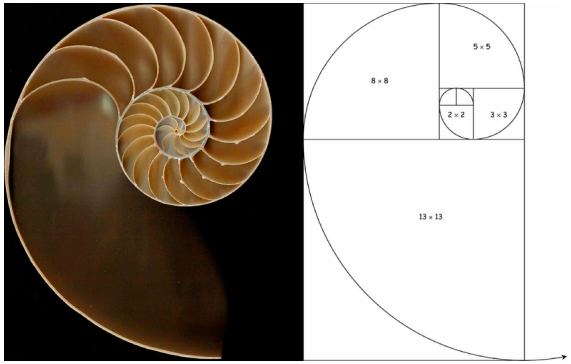
Fibonacci sequence

A spiral created by drawing circular arcs connecting the opposite corners of squares in a tiling where side lengths correspond to successive Fibonacci numbers.



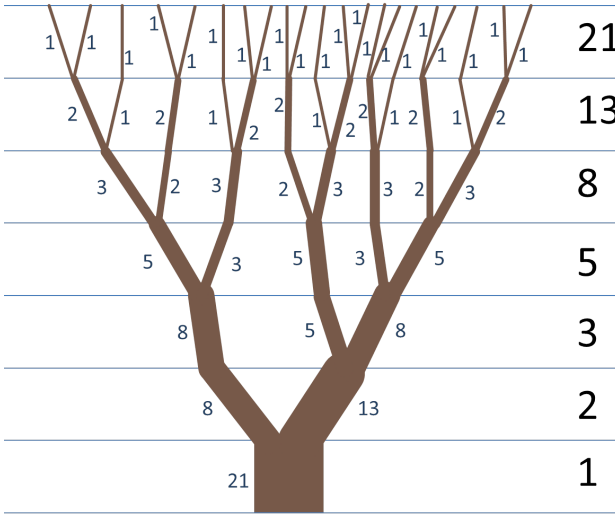
Fibonacci sequence in nature

- **Spiral Patterns**: The growth chambers of a nautilus shell create a spiral that approximately follows the Fibonacci sequence and the golden ratio.



Fibonacci sequence in nature

- **Spiral Patterns**: The growth chambers of a nautilus shell create a spiral that approximately follows the Fibonacci sequence and the golden ratio.
- **Branches**: Tree branches sometimes follow the Fibonacci sequence. The main branch splits into two, then one of those branches splits while the other stays dormant.

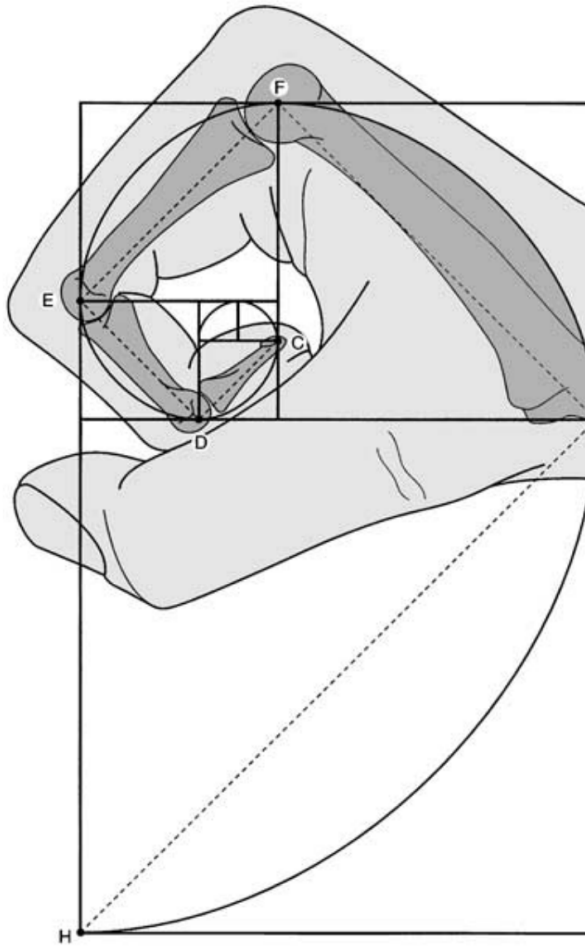


Fibonacci sequence in nature

- **Spiral Patterns**: The growth chambers of a nautilus shell create a spiral that approximately follows the Fibonacci sequence and the golden ratio.
- **Branches**: Tree branches sometimes follow the Fibonacci sequence. The main branch splits into two, then one of those branches splits while the other stays dormant.
- **Flower Petals:** The number of petals on many flowers corresponds to Fibonacci numbers: lilies (3 petals), buttercups (5 petals), and daisies (often 34 or 55 petals).

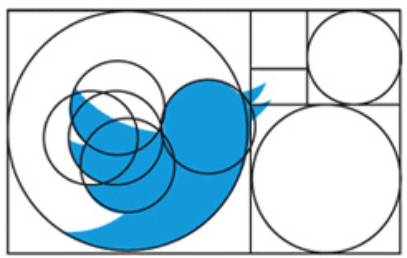
Fibonacci sequence in nature

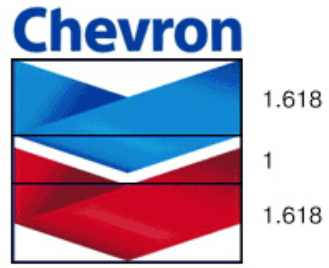
- **Spiral Patterns**: The growth chambers of a nautilus shell create a spiral that approximately follows the Fibonacci sequence and the golden ratio.
- **Branches**: Tree branches sometimes follow the Fibonacci sequence. The main branch splits into two, then one of those branches splits while the other stays dormant.
- **Flower Petals:** The number of petals on many flowers corresponds to Fibonacci numbers: lilies (3 petals), buttercups (5 petals), and daisies (often 34 or 55 petals).
- **Why?** Fibonacci patterns in nature are likely related to optimal growth and packing efficiency.
- **While the Fibonacci sequence appears in nature, it is not a universal rule.**

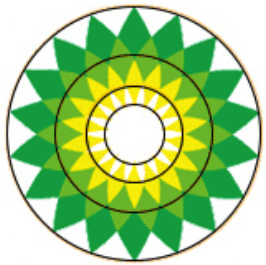


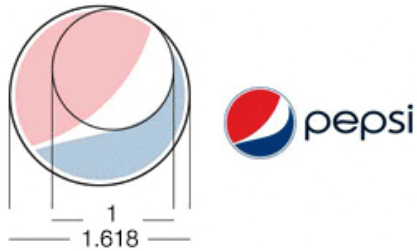
The Fibonacci sequence is extensively used in the arts

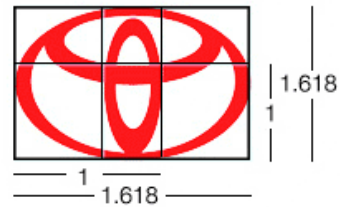
Golden ratio in famous logos











Fibonacci sequence tasks as Python refresher

We will use the Fibonacci sequence to review some basic concepts in Python. Basic experience with programming is expected.

We will cover:

- Loop control statements
- Vector and matrix indexing (using NumPy)
- Functions
- Plotting (using Matplotlib)

Task 1.1: Loop control statements

With loop control statements, you can repeatedly execute a block of code. There are two main types of loops:

1. **for** statements loop a specific number of times (often iterating over a sequence).
2. **while** statements loop as long as a condition remains true.

Your tasks:

1. Generate the first 20 elements of the Fibonacci sequence using a **for** loop.
2. Generate all Fibonacci numbers smaller than 500 using a **while** loop.

Note: Save generated numbers to a list or array; we will need them for the next task.

Solution 1.1.1: First 20 Fibonacci numbers

Generate the first 20 elements of the Fibonacci sequence using a **for** loop.

```
1  import numpy as np
2
3  n = 20
4  fib_seq = [1, 1]
5
6  for i in range(2, n):
7      fib_seq.append(fib_seq[i-1] + fib_seq[i-2])
8
9  print(fib_seq)
```

Solution 1.1.2: Fibonacci numbers smaller than 500

Generate all Fibonacci numbers smaller than 500 using a **while** loop.

```
fib_seq = [1, 1]
max_value = 5000
i = 1 # Python uses 0-based indexing

while (fib_seq[i] + fib_seq[i-1]) < max_value:
    fib_seq.append(fib_seq[i] + fib_seq[i-1])
    i += 1

print(fib_seq)
```

Task 1.2: Vector Indexing

Vector indexing allows you to access and manipulate specific elements within an array. We will use the **NumPy** library for this.

- **Basic Indexing (0-based)**
  - Access elements by their position: `x[2]` gets the *third* element of x.
  - Negative indexing: `x[-1]` gets the last element.
  - Slicing: `x[1:5]` gets elements from index 1 up to (but not including) index 5.
- **Boolean (Logical) Indexing**:
  - Create a boolean mask to filter elements: `x[x > 10]` extracts elements greater than 10.

Task 1.2: Vector Indexing

Your tasks (Assume `fib_seq` is a NumPy array):

- Divide the 6^{th} and 5^{th} element of the array.
- Divide the last and 2nd to last element of the array.
- Get all Fibonacci numbers smaller than 50.
- Get all Fibonacci numbers between 50 and 500.
- With the help of indexing and the modulus operator, replace all even Fibonacci numbers in the array with 0.

Task 1.2: Solution

- Divide 6^{th} and 5^{th} element of the array.

```
fib_seq[5] / fib_seq[4]
```

- Divide last and 2nd last element of the array.

```
fib_seq[-1] / fib_seq[-2]
```

- Get all Fibonacci numbers smaller than 50.

```
fib_seq[fib_seq < 50]
```

- Get all Fibonacci numbers between 50 and 500.

```
fib_seq[(fib_seq > 50)& (fib_seq < 500)]
```

- Replace all even Fibonacci numbers in the vector with 0.

```
fib_seq[fib_seq % 2 == 0] = 0
```

Matrix indexing is similar to vector indexing

- Define matrix (using NumPy)

```
M = np.array([[1,2,3], [4,5,6], [7,8,9]])
```

- Extract the element in row 2, column 3

```
M[1, 2]
```

- One or both of the row and column subscripts can be slices:

```
M[0:3, 1:3]
```

- A ":" selects the entire dimension

```
M[:, 1:3]
```

- To treat M as a flat vector (similar to linear indexing), use flatten:

```
M.flatten()[7]
```

- You can use a boolean array as a mask.

```
N = np.array([[1.3, 2.0], [4.8, 5.0]])
N[N % 1 == 0]
N[N > 3]
```

Task 1.3: Functions background

- Functions are reusable blocks of code that take inputs, perform calculations, and return outputs. They are essential for code organization and modularity.
- Functions in Python are defined using the `def` keyword.

```
# declare function
def circle_area(radius):
    pi = 3.14159 # approx. value of pi
    area = pi * radius**2
    return area

# call function
my_circle = circle_area(5)
```

Task 1.3: Functions

Your tasks

1. Write a function to return a list/array with generated n Fibonacci numbers.
2. Write a function to return a random Fibonacci number.
3. Write a function to check if a number is a Fibonacci number.

Think about your solution first; after that feel free to search for hints online.

Solution 1.3.1: Fibonacci generator

- We will reuse our previous Task 1.1 code.
- Handling edge cases (n=0, n=1) is important.
- **What are the limitations of this code?**

```
1  def generate_fibonacci(n):
2      fib_seq = [1, 1]
3      for i in range(2, n):
4          fib_seq.append(fib_seq[i-1] + fib_seq[i-2])
5      return fib_seq[:n]
6
7  print(generate_fibonacci(10))
```

Solution 1.3.2: Random Fibonacci number

```python
import random

def random_fibonacci(n):
    fib_seq = [1, 1]
    for i in range(2, n):
        fib_seq.append(fib_seq[i-1] + fib_seq[i-2])

    fib_seq = fib_seq[:n]
    return random.choice(fib_seq)

print(random_fibonacci(10))
```

Solution 1.3.3: Check if number is Fibonacci

One possible approach: Check if the last generated number before overstepping the threshold is the number in question.

```
def check_fibonacci(val):
    a, b = 1, 1

    while b < val:
        memory = b
        b = a + b
        a = memory

    return b == val

# Example usage:
# print(check_fibonacci(8)) # True
```

Task 4: Plotting

Plotting allows you to represent your data graphically. Python uses the library **Matplotlib** for this.
Your tasks:

1. Plot the first 15 elements of the Fibonacci sequence.
2. Calculate and plot the ratio of `n[i+1] / n[i]` for the first 15 elements of the Fibonacci sequence.

Solutions 1.4

1. Plot the first 15 elements of the Fibonacci sequence:

```
plt.plot(fib_seq_15, '-s')
plt.show()

# Calculate ratio (F_n / F_{n-1})
ratio = fib_seq_15[1:] / fib_seq_15[:-1]
print(ratio)
```

2. Calculate and plot the ratio of `n[i+1] / n[i]` in the first 15 elements.

```
plt.figure()
plt.plot(ratio, '-s')
plt.show()
```

Section 2: Malthusian Growth Model

Malthusian Growth Model

- The Malthusian growth model is a mathematical model of population growth.
- It essentially describes exponential population growth. [Image of exponential growth curve]
- The model is named after Thomas Malthus, who wrote *An Essay on the Principle of Population* (1798), one of the first and most influential books on population dynamics.

Thomas Robert Malthus (1766 - 1834), an English economist and scholar.

Key assumptions of Thomas Malthus

- **Population grows exponentially when resources are abundant.**
- Malthus further assumed that agricultural production increases only linearly (arithmetically). [Image of Malthusian catastrophe graph showing exponential population vs linear resources]
- Population growth is limited by available resources and can be reduced by famine, disease, or war.

Malthusian Catastrophe

A situation where the population exceeds the agricultural production of its environment. This results in severe consequences like poverty, famine, disease, and war. Malthus believed a Malthusian catastrophe was inevitable unless population growth was controlled.

Malthus, Darwin, and Natural Selection

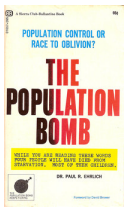
- Before reading Malthus, Darwin thought that living things reproduced just enough individuals to keep populations stable.
- Malthus helped him realize that as populations breed beyond their means, advantageous traits become more prevalent due to competition for resources.
- This led to the development of the theory of natural selection.

"In October 1838... I happened to read for amusement Malthus on Population, and being well prepared to appreciate the struggle for existence... ***it at once struck me that under these circumstances favourable variations would tend to be preserved, and unfavourable ones to be destroyed.*** *The result of this would be the formation of a new species."*

— Charles Darwin

Neo-Malthusianism

- A revival of Malthusian concerns in the 20th century, often focused on environmental limits.
- Neo-Malthusians advocate for population control due to concerns about overpopulation, resource depletion, and environmental degradation.
- Widely discussed in the 1960s; a famous example is Paul Ehrlich's book *The Population Bomb* (1968).
- Their influence can be seen in family planning initiatives and advocacy for sustainability.
- Modern environmentalism acknowledges the impact of population growth on resource use, pollution, and environmental strain.



Malthusian Growth Model Equation

The change in population dP/dt is proportional to the current population size P at time t. The parameter r is the intrinsic rate of increase (the difference between birth rate and death rate).

$$\frac{dP}{dt} = rP$$

To solve the differential equation, we separate the variables. We move terms with P to one side and terms with t to the other side.

$$\frac{dP}{P} = rdt$$

Malthusian Growth Model Equation

We integrate both sides of the equation. Integration introduces a constant of integration, C.

$$\int \frac{dP}{P} = \int r dt$$
$$\ln |P| = rt + C$$

We exponentiate both sides. We remove the absolute value as the population size is always positive.

$$e^{\ln P} = e^{rt+C}$$
$$P = e^{C} e^{rt}$$

The integration constant e^{C} represents the initial population P_0. Substitution leads to the final explicit form:

$$P(t) = P_0 e^{rt}$$

Numerical vs Explicit Solution

- Numerical and explicit solutions are two methods used to solve mathematical problems in modeling and simulation.

- **Explicit (Analytical) Solution**: A mathematically **exact** solution expressed as a formula.
  - Provides a precise value for any time t.
  - Often allows for a deeper understanding of the underlying mathematical relationships.
  - It can be very difficult or impossible to find explicit solutions for complex, non-linear problems.

Numerical vs Explicit Solution

- Numerical and explicit solutions are two methods used to solve mathematical problems in modeling and simulation.

- **Numerical Solution**: Uses **iterative methods to approximate** a solution using computational steps. [Image of Euler method steps visualization]
  - Allows solving a much wider range of complex problems (e.g., where parameters change over time).
  - Solutions are only approximate.
  - Solutions can be affected by cumulative calculation errors (discretization error).
  - Difficult to validate without real-world data or an explicit solution for comparison.

Numerical vs Explicit Solution for Malthusian Growth

Explicit Solution
Calculation using the formula:

$$P(t) = P_0 e^{rt}$$

Where:

- P_0 is the initial population at time 0
- r is the growth rate
- t is the time

Numerical Solution (Euler's Method):

$$\frac{dP}{dt} \approx \frac{\Delta P}{\Delta t}$$

Algorithm:

1. **Discretization:**

$$t_{i+1} = t_i + \Delta t$$

2. **Iteration (Update Rule):**

$$P(t_{i+1}) = P(t_i) + rP(t_i)\Delta t$$

Where:

- $P(t_i)$ is the population at time step i
- Δt is the time step size

Task 2.1: Population at discrete times

Calculate population at certain times of interest using the explicit formula.

- Initial population size (P_0): 2.5 million
- Yearly growth rate (r): 2.7%
- Equation: $P(t) = P_0 e^{rt}$
- Time in years (t):
  - 1
  - 5
  - 7.6
  - 25.78
- Note: Population size and growth rate correspond to the population of Senegal in 1950.

Solution 2.1: Population at discrete times

```python
import numpy as np

P0 = 2.5    # Initial population
r = 0.027   # Growth rate 2.7%

t_task1 = np.array([1, 5, 7.5, 25.78])

print("Years of interest:")
print(t_task1)
print("Calculated population size:")
print(P0 * np.exp(r * t_task1))
```

Solution 2.1: Population at discrete times

- The results show population size at times of 1, 5, 7.5, and 25.78 years.
- The explicit solution allows us to easily calculate population predictions at any given specific time without iterating.

```
Years of interest:
    [1, 5, 7.5, 25.78]
Calculated population size (Millions):
    [2.568, 2.861, 3.061, 5.014]
```

Task 2.2: Continuous modeling with explicit solution

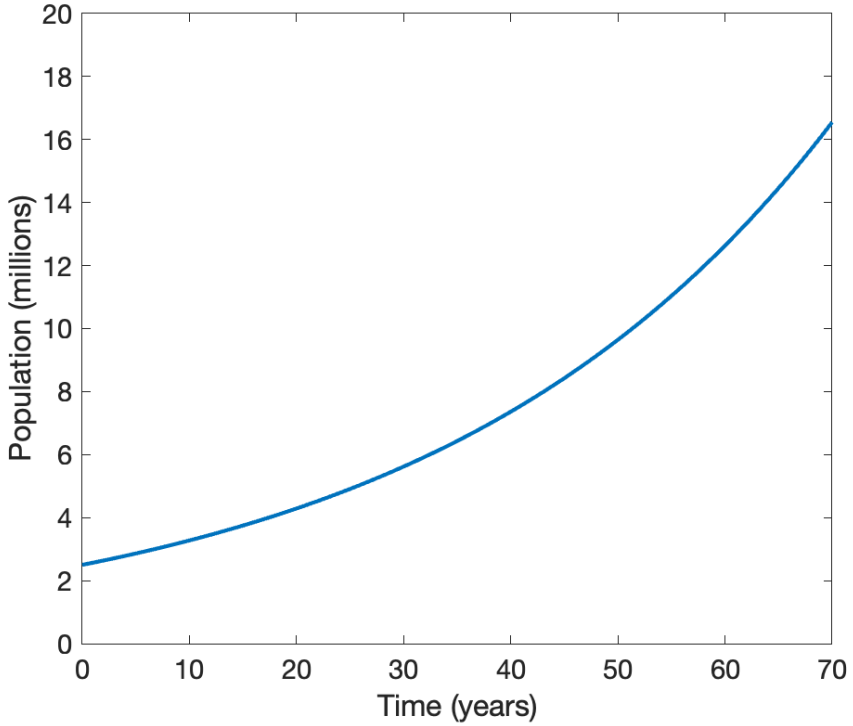
- The population has the same parameters as in Task 2.1.
- Calculate the population for the next 70 years.
- The time step is 1 year.
- Plot the results.

Solution 2.2: Continuous modeling with explicit solution

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  P0 = 2.50
5  r = 0.027
6  t_start, t_end, t_diff = 0, 70, 1
7
8  t_expl = np.arange(t_start, t_end + t_diff, t_diff)
9  P_expl = P0 * np.exp(r * t_expl)
10
11 plt.plot(t_expl, P_expl, linewidth=2)
12 plt.xlabel("Time (years)")
13 plt.ylabel("Population (millions)")
14 plt.ylim(0, 20)
15 plt.show()
```

Solution 2.2: Continuous modeling with explicit solution

Population increases exponentially.



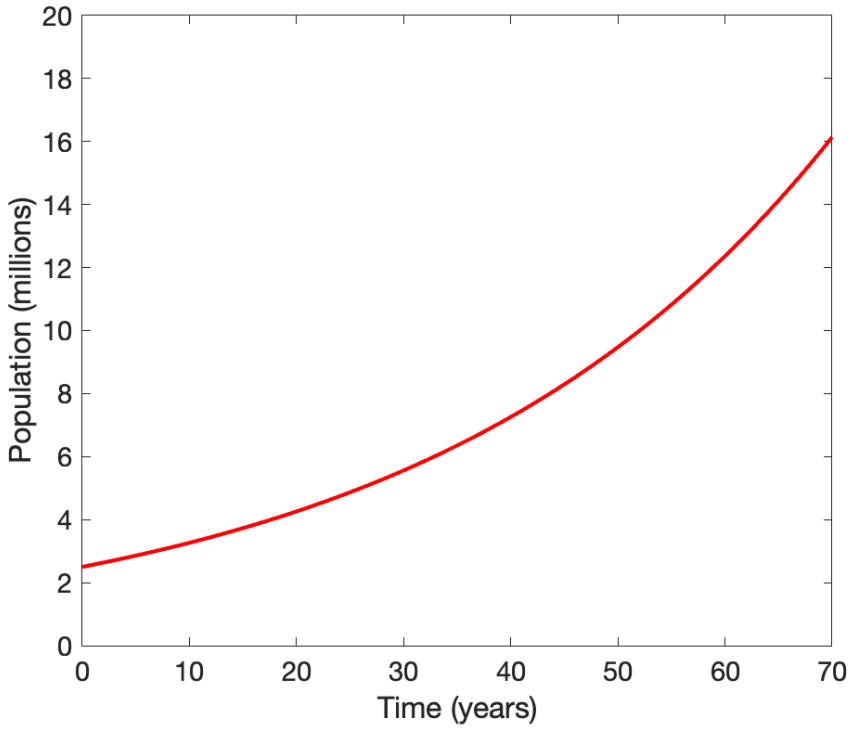
Task 2.3: Numerical approach

- Use the numerical approach (Euler's method) to calculate population from year 0 to 70.
- The time step should be 1 year.
- Plot the result.

Solution 2.3: Numerical approach

```python
import matplotlib.pyplot as plt

P0, r = 2.50, 0.027
t_start, t_end, t_diff = 0, 70, 1

t_num = [t_start]
P_num = [P0]

# Numerical calculation loop
for i in range(int(t_end/t_diff)):
    P_new = P_num[i] + r * P_num[i] * t_diff
    t_new = t_num[i] + t_diff
    P_num.append(P_new)
    t_num.append(t_new)

plt.plot(t_num, P_num, color="red")
plt.xlabel("Time (years)")
plt.ylabel("Population (millions)")
plt.ylim(0, 20)
plt.show()
```

Solution 2.3: Numerical approach



Task 2.4: Difference in solutions

- Create a plot that shows results from both explicit and numerical approaches and the difference (error).
- Add a start year of 1950 to the plot to represent the correct timeline.
- Compare the difference for time steps (Δt) of 1 year, 1 month, and 1 day.

Solution 2.4: The difference in solutions

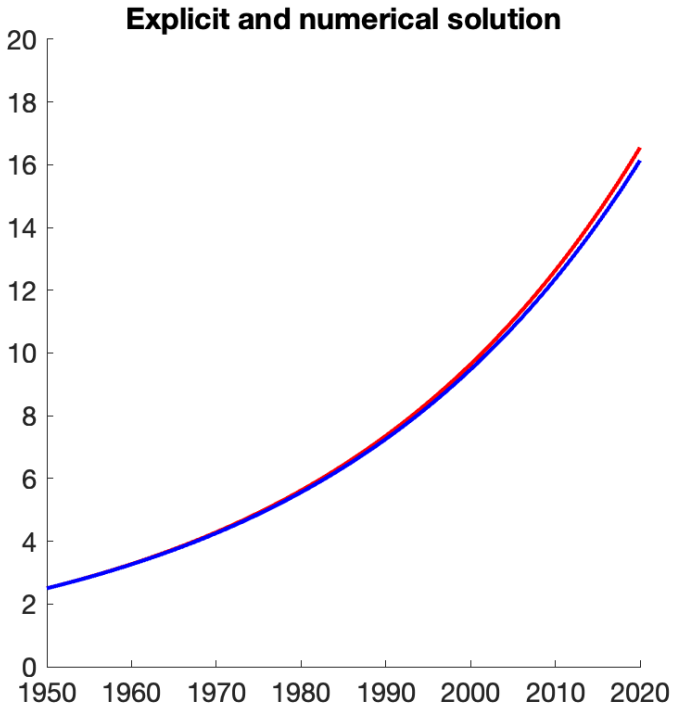
```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  # MODELING PARAMETERS
5  P0, r = 2.50,  0.027
6  start_year = 1950
7  t_start, t_end, t_diff = 0, 70, 1
8
9  # EXPLICIT SOLUTION
10 t_expl = np.arange(t_start, t_end + t_diff, t_diff)
11 P_expl = P0 * np.exp(r * t_expl)
12
13 # NUMERICAL SOLUTION
14 t_num = [t_start]
15 P_num = [P0]
16 for i in range(int(t_end/t_diff)):
17     P_num.append(P_num[i] + r * P_num[i] * t_diff)
18     t_num.append(t_num[i] + t_diff)
19 P_num = np.array(P_num)
20 t_num = np.array(t_num)
```

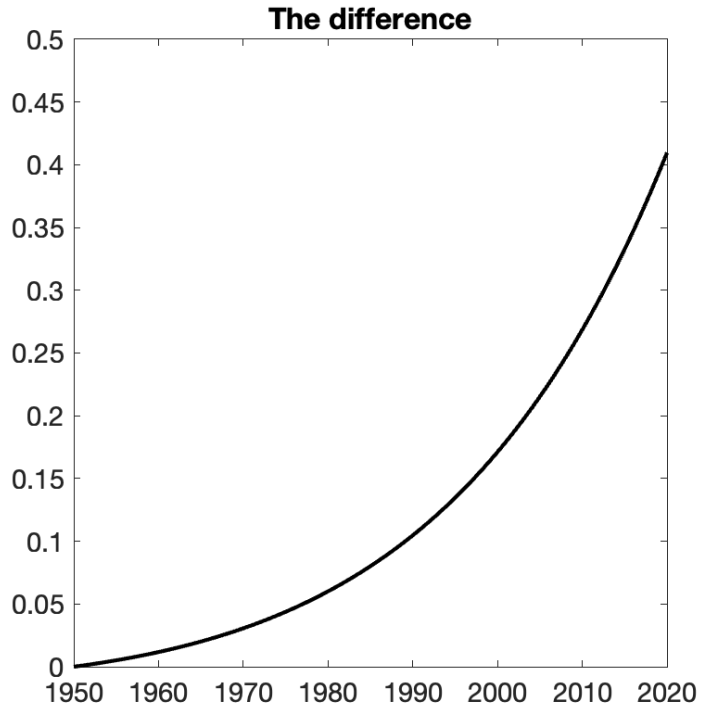
Solution 2.4: The difference in solutions

```
# PLOTTING
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

# Plot 1: Explicit vs Numerical
ax1.plot(t_expl + start_year, P_expl, color="red",
    label="Explicit")
ax1.plot(t_num + start_year, P_num, color="blue",
    label="Numerical")
ax1.set_title("Explicit and numerical solution")
ax1.set_ylim(0, 20)
ax1.set_xlim(1950, 2020)
ax1.legend()

# Plot 2: Difference
ax2.plot(t_expl + start_year, P_expl - P_num, color="black")
ax2.set_title("The difference")
ax2.set_ylim(0, 0.5)
ax2.set_xlim(1950, 2020)

plt.tight_layout()
plt.show()
```

Solution 2.4: The difference in solutions

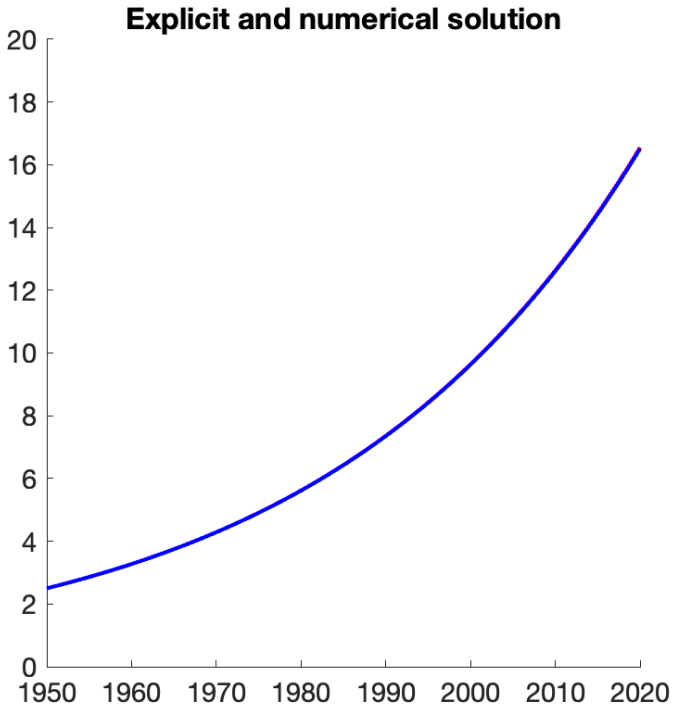
Step size of 1 year leads to substantial error.

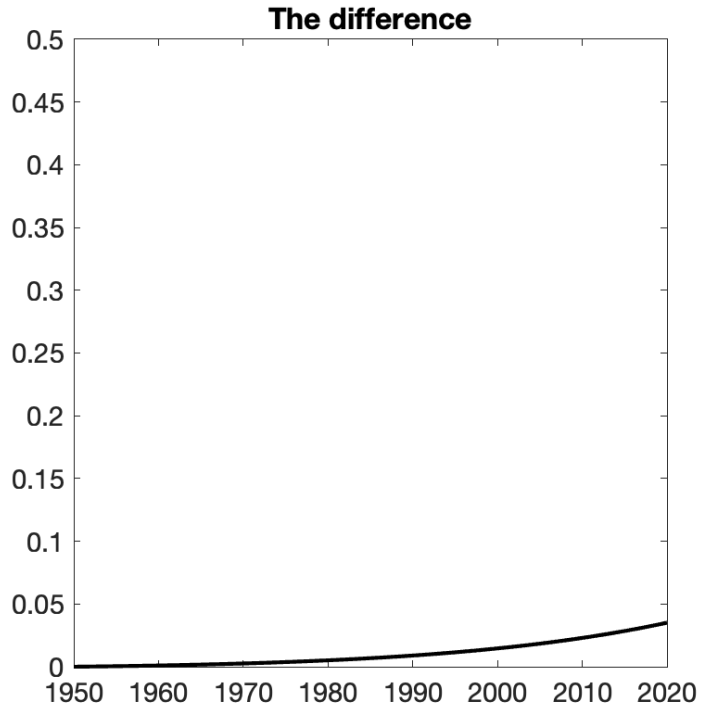




Solution 2.4: The difference in solutions

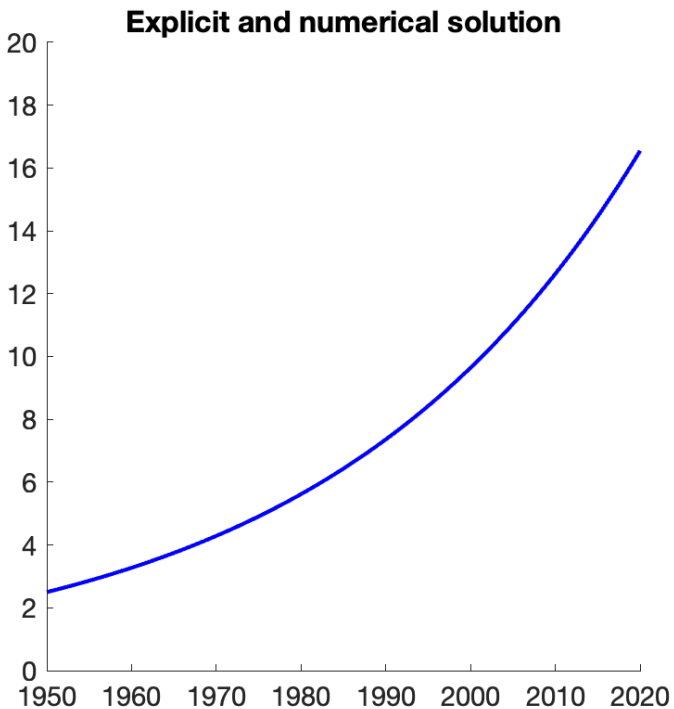
Step size of 1 month reduces the error significantly.

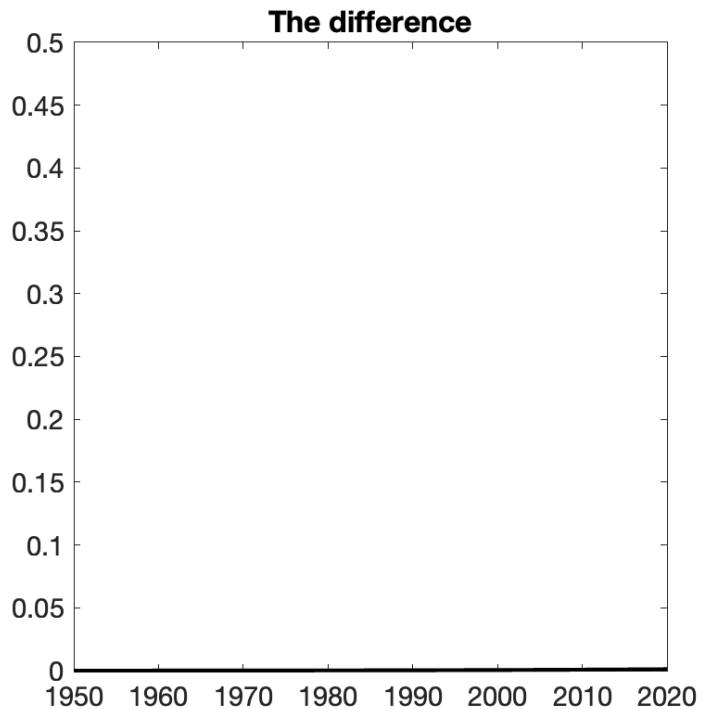




Solution 2.4: The difference in solutions

Step size of 1 day leads to negligible error.





Task 2.5: Real world data

Compare the prediction modeled by the explicit formula with real data of the Senegalese population.

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  # Growth rate
5  r = 0.027
6
7  # Start year
8  start_year = 1950
9
10 # Time axis (years)
11 t_sen = np.array([0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50,
       55, 60, 65, 70, 72, 73, 74])
12
13 # Senegalese population (millions)
```

Solution 2.5: Real world data

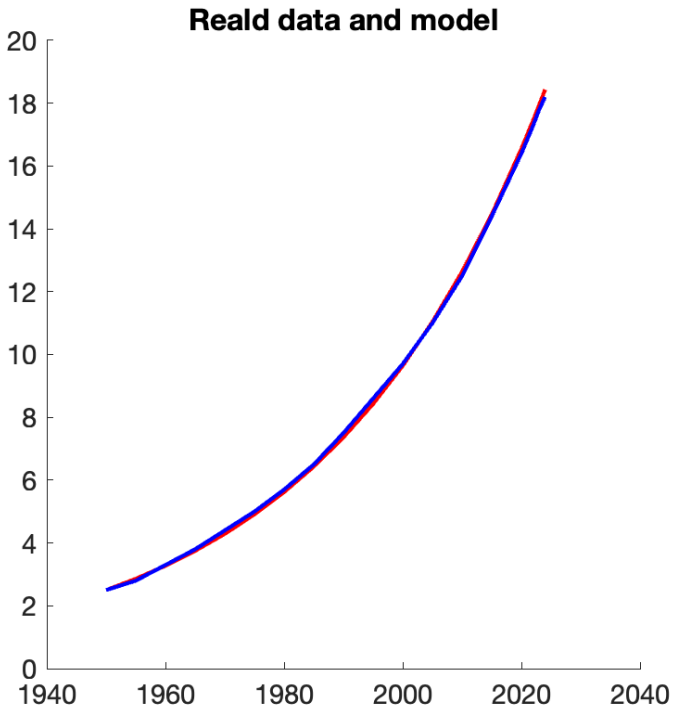
```python
import numpy as np
import matplotlib.pyplot as plt

# Growth rate
r = 0.027

# Start year
start_year = 1950

# Time axis (years)
t_sen = np.array([0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50,
    55, 60, 65, 70, 72, 73, 74])

# Senegalese population (millions)
P_sen = np.array([2.5, 2.8, 3.3, 3.8, 4.4, 5.0, 5.7, 6.5,
    7.5, 8.6, 9.7, 11.0, 12.5, 14.4, 16.4, 17.3, 17.8,
    18.2])

# EXPLICIT SOLUTION
# Initial population is the first value: P_sen[0]
P_expl = P_sen[0] * np.exp(r * t_sen)
```

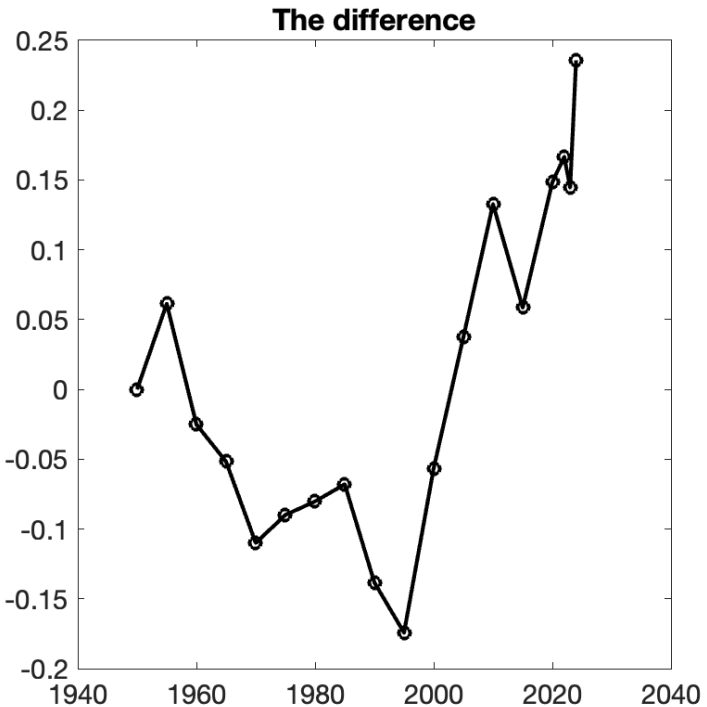
Solution 2.5: Real world data

```
1
2  # PLOTTING
3  fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
4
5  # Tile 1: Real data and model
6  ax1.plot(t_sen + start_year, P_expl, color="red",
       label='Model')
7  ax1.plot(t_sen + start_year, P_sen, color="blue",
       label='Real Data')
8  ax1.set_title("Real data and model")
9  ax1.set_ylim(0, 20)
10 ax1.legend()
11
12 # Tile 2: The difference
13 ax2.plot(t_sen + start_year, P_expl - P_sen, '-o',
       color="black")
14 ax2.set_title("The difference")
15
16 plt.tight_layout()
17 plt.show()
```

Solution 2.5: Real world data

Even a very simple model can capture reality very well.





Task 2.6: Real world data — USA

- Compare the prediction modeled by the explicit formula for the USA with real data.
- Adjust the time span, start year, and initial population.
- Set growth rate to 2.75% (historical average for that specific start period).

```
# Growth rate
r = 0.027

# Start year
start_year = 1950

# Time axis (years)
t_sen = np.array([0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50,
    55, 60, 65, 70, 72, 73, 74])

# Senegalese population (millions)
P_sen = np.array([2.5, 2.8, 3.3, 3.8, 4.4, 5.0, 5.7, 6.5,
    7.5, 8.6, 9.7, 11.0, 12.5, 14.4, 16.4, 17.3, 17.8,
    18.2])

# EXPLICIT SOLUTION
```

Solution 2.6: Real world data — USA

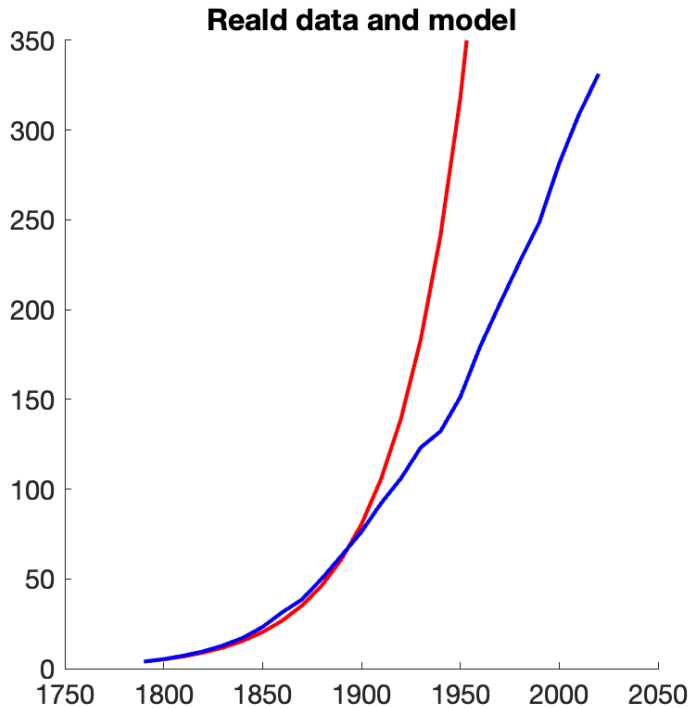
```python
import numpy as np
import matplotlib.pyplot as plt

# Growth rate
r = 0.027

# Start year
start_year = 1950

# Time axis (years)
t_sen = np.array([0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50,
    55, 60, 65, 70, 72, 73, 74])

# Senegalese population (millions)
P_sen = np.array([2.5, 2.8, 3.3, 3.8, 4.4, 5.0, 5.7, 6.5,
    7.5, 8.6, 9.7, 11.0, 12.5, 14.4, 16.4, 17.3, 17.8,
    18.2])

# EXPLICIT SOLUTION
# Initial population is the first value: P_sen[0]
P_expl = P_sen[0] * np.exp(r * t_sen)

# PLOTTING
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
```

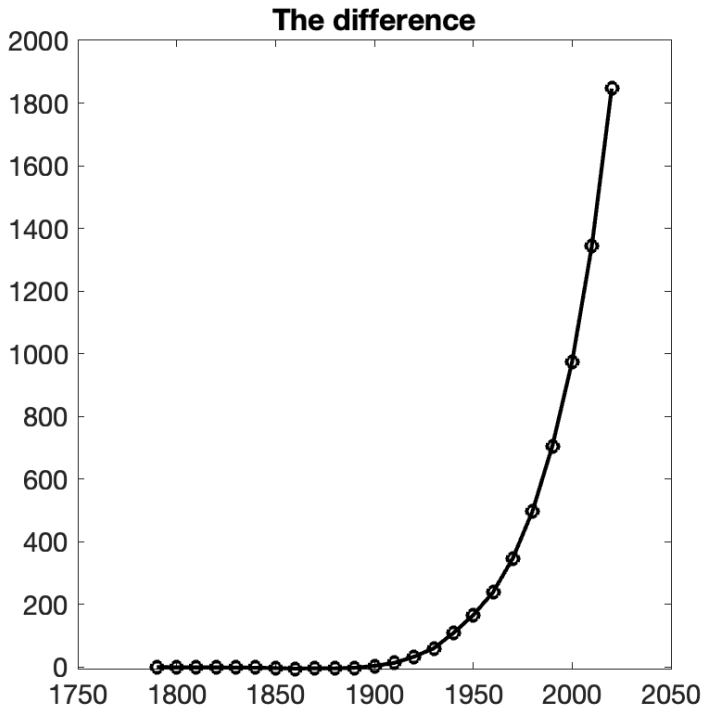
Solution 2.6: Real world data — USA

```
1
2  # Tile 1: Real data and model
3  ax1.plot(t_sen + start_year, P_expl, color="red",
       label='Model')
4  ax1.plot(t_sen + start_year, P_sen, color="blue",
       label='Real Data')
5  ax1.set_title("Real data and model")
6  ax1.set_ylim(0, 20)
7  ax1.legend()
8
9  # Tile 2: The difference
10 ax2.plot(t_sen + start_year, P_expl - P_sen, '-o',
       color="black")
11 ax2.set_title("The difference")
12
13 plt.tight_layout()
14 plt.show()
```

Solution 2.6: Real world data — USA

The model is reasonably precise for the first one hundred years, then it diverges.
Why? (The growth rate slowed down; resources/space became less "unlimited").





Section 3: Logistic Growth Model

Logistic growth

The **logistic growth model represents how populations change over time when resources are limited**. The logistic model incorporates a carrying capacity—the maximum population size a specific environment can sustainably support.

Logistic growth

The **logistic growth model represents how populations change over time when resources are limited**. The logistic model incorporates a carrying capacity—the maximum population size a specific environment can sustainably support.

Key Concepts

- **Carrying Capacity (K)**: A ceiling that limits population growth due to limited resources (food, space, water, etc.).

Logistic growth

The **logistic growth model represents how populations change over time when resources are limited**. The logistic model incorporates a carrying capacity—the maximum population size a specific environment can sustainably support.

Key Concepts

- **Carrying Capacity (K)**: A ceiling that limits population growth due to limited resources (food, space, water, etc.).
- **Intrinsic Growth Rate (r)**: The rate at which a population would grow if there were unlimited resources.

Logistic growth

The **logistic growth model represents how populations change over time when resources are limited**. The logistic model incorporates a carrying capacity—the maximum population size a specific environment can sustainably support.

Key Concepts

- **Carrying Capacity (K)**: A ceiling that limits population growth due to limited resources (food, space, water, etc.).
- **Intrinsic Growth Rate (r)**: The rate at which a population would grow if there were unlimited resources.
- **S-Shaped Curve**: The logistic growth model plots as an S-shaped curve. Growth is initially exponential, then slows as it approaches the carrying capacity.

Equation of logistic growth

Differential equation of logistic growth

$$\frac{dP}{dt} = rP\left(1 - \frac{P}{K}\right)$$

Equation of logistic growth

Differential equation of logistic growth

$$\frac{dP}{dt} = rP\left(1 - \frac{P}{K}\right)$$

Iterative Form:

$$P(t_{i+1}) = P(t_i) + rP(t_i)\left(1 - \frac{P(t_i)}{K}\right)\Delta t$$

Equation of logistic growth

Differential equation of logistic growth

$$\frac{dP}{dt} = rP\left(1 - \frac{P}{K}\right)$$

Iterative Form:

$$P(t_{i+1}) = P(t_i) + rP(t_i)\left(1 - \frac{P(t_i)}{K}\right)\Delta t$$

Explicit solution

$$P(t) = \frac{K}{1 + \left(\frac{K - P_0}{P_0}\right)e^{-rt}}$$

Equation of logistic growth

Differential equation of logistic growth

$$\frac{dP}{dt} = rP\left(1 - \frac{P}{K}\right)$$

Iterative Form:

$$P(t_{i+1}) = P(t_i) + rP(t_i)\left(1 - \frac{P(t_i)}{K}\right)\Delta t$$

Explicit solution

$$P(t) = \frac{K}{1 + \left(\frac{K - P_0}{P_0}\right)e^{-rt}}$$

where

- $P(t_i)$ is the population at time t_i

Equation of logistic growth

Differential equation of logistic growth

$$\frac{dP}{dt} = rP\left(1 - \frac{P}{K}\right)$$

Iterative Form:

$$P(t_{i+1}) = P(t_i) + rP(t_i)\left(1 - \frac{P(t_i)}{K}\right)\Delta t$$

Explicit solution

$$P(t) = \frac{K}{1 + \left(\frac{K - P_0}{P_0}\right)e^{-rt}}$$

where

- $P(t_i)$ is the population at time t_i
- P_0 is the initial population

Equation of logistic growth

Differential equation of logistic growth

$$\frac{dP}{dt} = rP\left(1 - \frac{P}{K}\right)$$

Iterative Form:

$$P(t_{i+1}) = P(t_i) + rP(t_i)\left(1 - \frac{P(t_i)}{K}\right)\Delta t$$

Explicit solution

$$P(t) = \frac{K}{1 + \left(\frac{K - P_0}{P_0}\right)e^{-rt}}$$

where

- $P(t_i)$ is the population at time t_i
- P_0 is the initial population
- r is the growth rate

Equation of logistic growth

Differential equation of logistic growth

$$\frac{dP}{dt} = rP\left(1 - \frac{P}{K}\right)$$

Iterative Form:

$$P(t_{i+1}) = P(t_i) + rP(t_i)\left(1 - \frac{P(t_i)}{K}\right)\Delta t$$

Explicit solution

$$P(t) = \frac{K}{1 + \left(\frac{K - P_0}{P_0}\right)e^{-rt}}$$

where

- $P(t_i)$ is the population at time t_i
- P_0 is the initial population
- r is the growth rate
- K is the carrying capacity

Equation of logistic growth

Differential equation of logistic growth

$$\frac{dP}{dt} = rP\left(1 - \frac{P}{K}\right)$$

Iterative Form:

$$P(t_{i+1}) = P(t_i) + rP(t_i)\left(1 - \frac{P(t_i)}{K}\right)\Delta t$$

Explicit solution

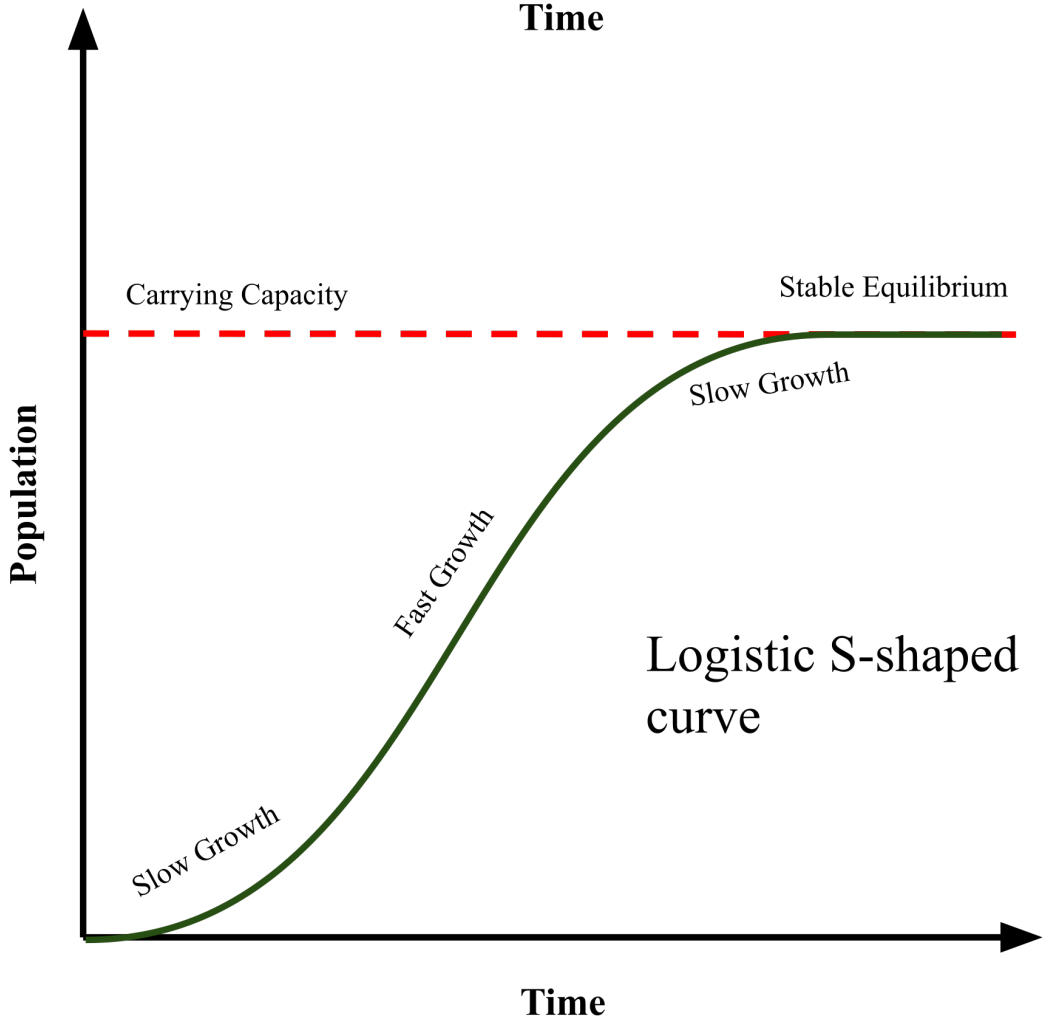
$$P(t) = \frac{K}{1 + \left(\frac{K - P_0}{P_0}\right)e^{-rt}}$$

where

- $P(t_i)$ is the population at time t_i
- P_0 is the initial population
- r is the growth rate
- K is the carrying capacity
- t is time, Δt is the time step

Logistic growth of population over time

Figure 1: Logistic Growth of Population Size Over Time



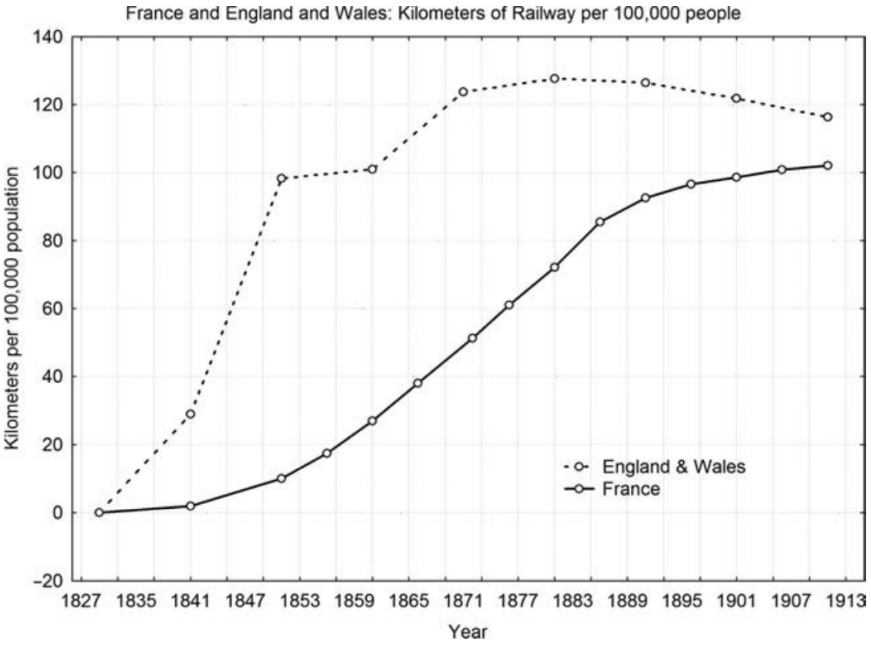
Carrying capacity is the amount of organisms within a region that the environment can support sustainably

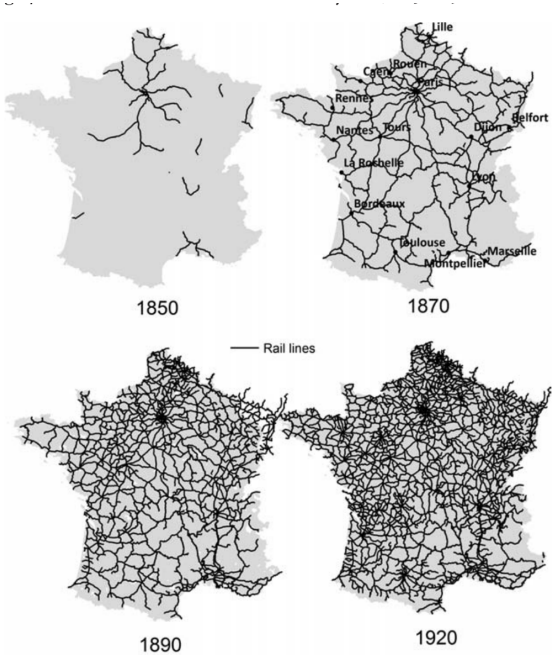
Stable equilibrium is met when the population aligns with the carrying capacity line

Slow growth occurs when natality is slightly above mortality, for fast growth natality is drastically greater than mortality

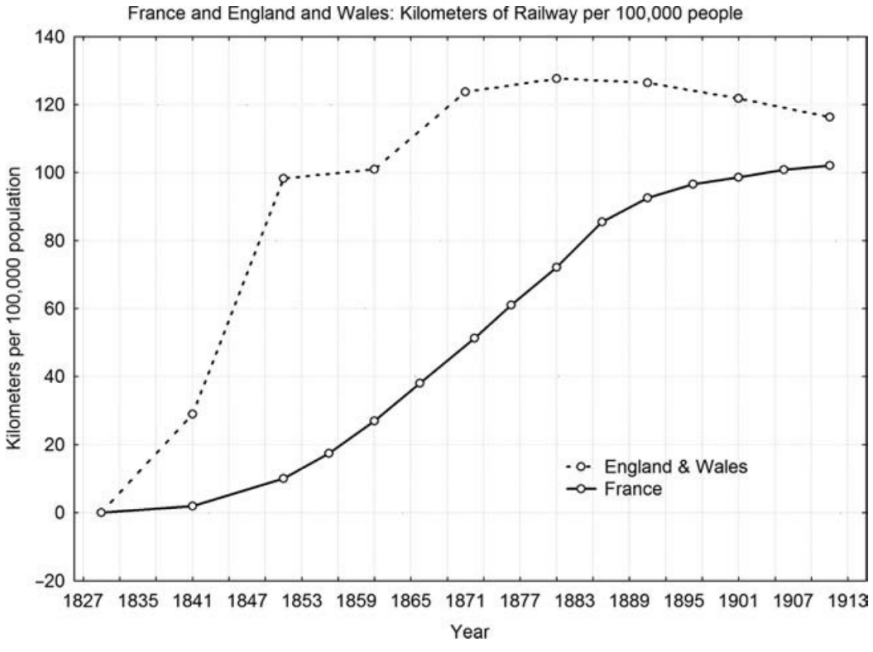
The S-shaped logistic curve is formed when growth rate decreases as carrying capacity is approached by the population

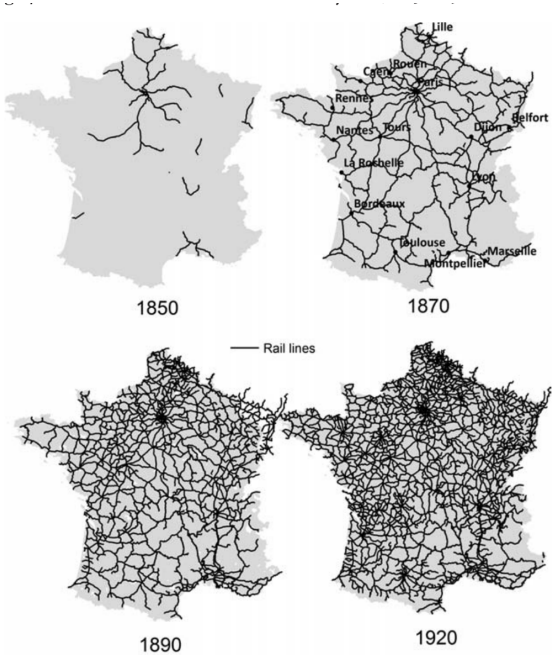
Example — French and English railway construction





Example — French and English railway construction

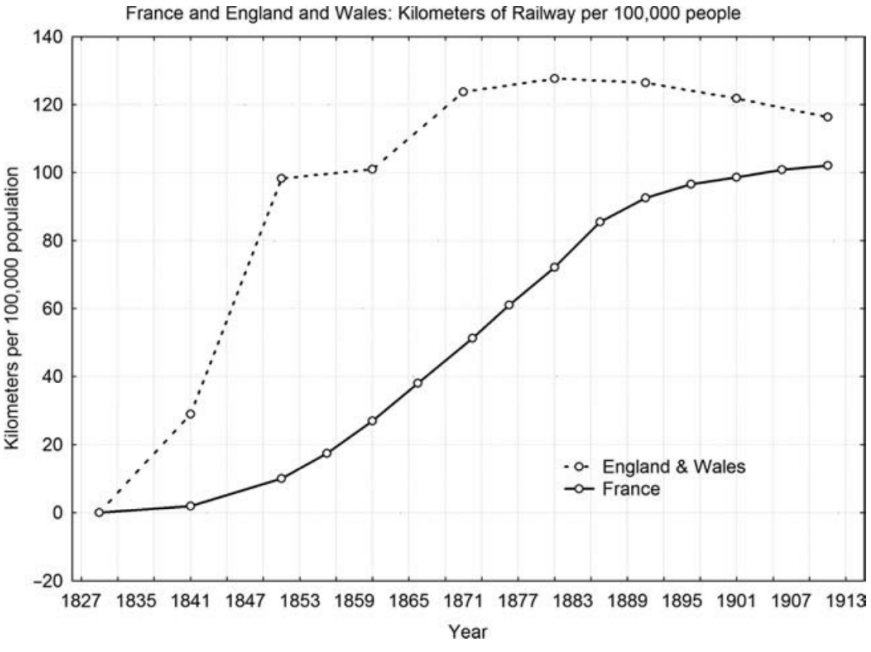


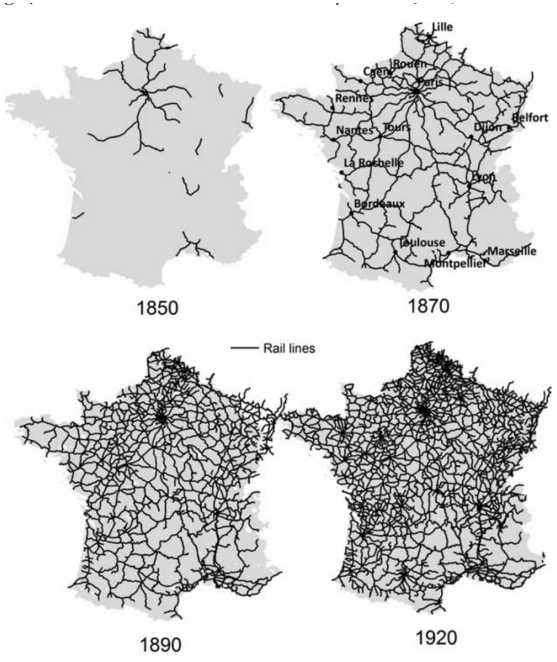


Topics for discussion

- Why do you think that the French railway construction follows logistic growth but the English does not?

Example — French and English railway construction

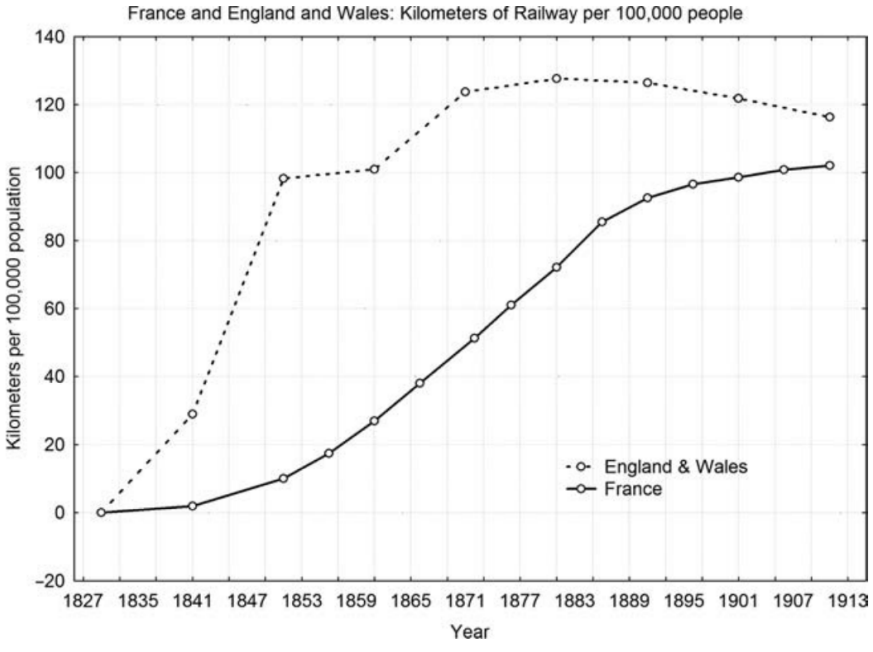


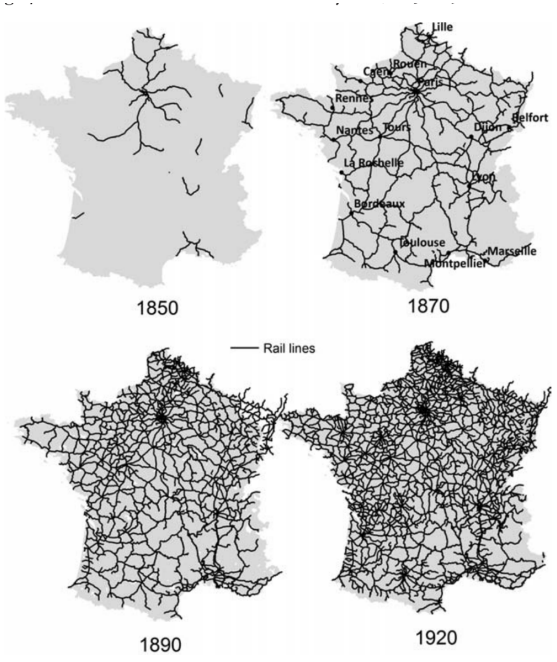


Topics for discussion

- Why do you think that the French railway construction follows logistic growth but the English does not?
- How would you explain **carrying capacity** in the context of railroad construction?

Example — French and English railway construction





Topics for discussion

- Why do you think that the French railway construction follows logistic growth but the English does not?
- How would you explain **carrying capacity** in the context of railroad construction?
- What may affect the carrying capacity for railways? Is the capacity constant or dynamic?

Closer look at logistic growth equation

Differential equation of logistic growth

$$\frac{dP}{dt} = rP\left(1 - \frac{P}{K}\right)$$

Let's explore the equation

- Try to figure out how the carrying capacity affects population growth. Try different combinations of P and K.

Closer look at logistic growth equation

Differential equation of logistic growth

$$\frac{dP}{dt} = rP\left(1 - \frac{P}{K}\right)$$

Let's explore the equation

- Try to figure out how the carrying capacity affects population growth. Try different combinations of P and K.
- What are the equilibrium solutions where there is no population growth?

Derivation of Equilibrium Solutions

To find equilibrium solutions, we set $\frac{dP}{dt} = 0$:

$$0 = rP\left(1 - \frac{P}{K}\right)$$

Derivation of Equilibrium Solutions

To find equilibrium solutions, we set $\frac{dP}{dt} = 0$:

$$0 = rP\left(1 - \frac{P}{K}\right)$$

This equation is satisfied when either:

- The population is zero

$$P = 0$$

Derivation of Equilibrium Solutions

To find equilibrium solutions, we set $\frac{dP}{dt} = 0$:

$$0 = rP\left(1 - \frac{P}{K}\right)$$

This equation is satisfied when either:

- The population is zero

$$P = 0$$

- The growth rate is zero

$$r = 0$$

Derivation of Equilibrium Solutions

To find equilibrium solutions, we set $\frac{dP}{dt} = 0$:

$$0 = rP\left(1 - \frac{P}{K}\right)$$

This equation is satisfied when either:

- The population is zero

$$P = 0$$

- The growth rate is zero

$$r = 0$$

- The population size equals the carrying capacity

$$1 - \frac{P}{K} = 0$$

$$1 = \frac{P}{K}$$

$$P = K$$

History

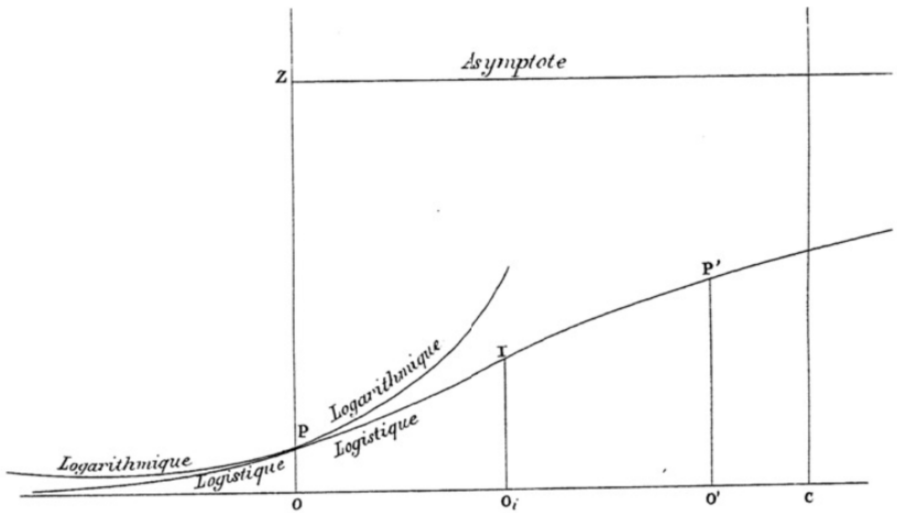
- **Pierre Verhulst**: A Belgian mathematician who first proposed the logistic model in the 1830s and 1840s. His model sought to refine the exponential population growth model of Malthus.

History

- **Pierre Verhulst**: A Belgian mathematician who first proposed the logistic model in the 1830s and 1840s. His model sought to refine the exponential population growth model of Malthus.
- Verhulst got inspiration from the concept of intraspecific competition—the idea that as populations get larger, individuals compete more intensely for limited resources.

History

- **Pierre Verhulst**: A Belgian mathematician who first proposed the logistic model in the 1830s and 1840s. His model sought to refine the exponential population growth model of Malthus.
- Verhulst got inspiration from the concept of intraspecific competition—the idea that as populations get larger, individuals compete more intensely for limited resources.
- This equation was published after Verhulst had read Thomas Malthus' *An Essay on the Principle of Population*.



Pierre Verhulst and the first plot of the logistic growth curve

The logistic growth model has extensive applications

- **Biology**: Used extensively in modeling animal, plant, and bacterial population growth.

The logistic growth model has extensive applications

- **Biology**: Used extensively in modeling animal, plant, and bacterial population growth.
- **Epidemiology and medicine**: Helps analyze how diseases spread within a population or tumor growth.

The logistic growth model has extensive applications

- **Biology**: Used extensively in modeling animal, plant, and bacterial population growth.
- **Epidemiology and medicine**: Helps analyze how diseases spread within a population or tumor growth.
- **Economics**: Modeling market adoption of new products or technologies—a diffusion of innovation.

The logistic growth model has extensive applications

- **Biology**: Used extensively in modeling animal, plant, and bacterial population growth.
- **Epidemiology and medicine**: Helps analyze how diseases spread within a population or tumor growth.
- **Economics**: Modeling market adoption of new products or technologies—a diffusion of innovation.
- **Linguistics**: An innovation that is at first marginal begins to spread more quickly with time, and then more slowly as it becomes more universally adopted.

The logistic growth model has extensive applications

- **Biology**: Used extensively in modeling animal, plant, and bacterial population growth.
- **Epidemiology and medicine**: Helps analyze how diseases spread within a population or tumor growth.
- **Economics**: Modeling market adoption of new products or technologies—a diffusion of innovation.
- **Linguistics**: An innovation that is at first marginal begins to spread more quickly with time, and then more slowly as it becomes more universally adopted.
- **Statistics and machine learning**: Logistic functions are often used in artificial neural networks to introduce nonlinearity in the model—but it is mostly for the shape.

Tasks overview

1. Numerically solve the logistic growth equation and plot:
   - Population size vs. time
   - Population increase vs. time
   - Per capita growth rate vs. population size
   - Population increase vs. population size

Tasks overview

1. Numerically solve the logistic growth equation and plot:
   - Population size vs. time
   - Population increase vs. time
   - Per capita growth rate vs. population size
   - Population increase vs. population size
2. Adjust the appearance of the plot:
   - Add a plot title and axis labels
   - Change line thickness and appearance
   - Experiment with other changes

Tasks overview

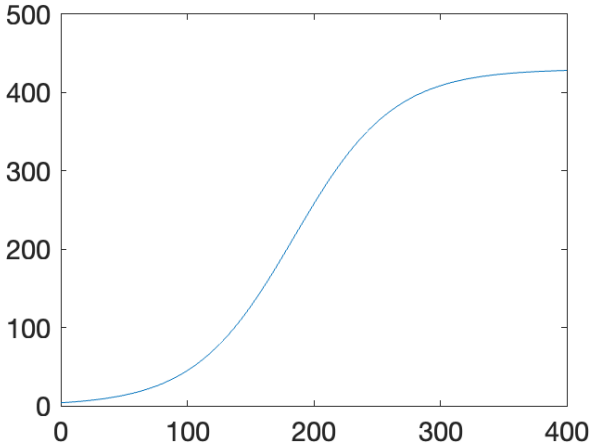
1. Numerically solve the logistic growth equation and plot:
   - Population size vs. time
   - Population increase vs. time
   - Per capita growth rate vs. population size
   - Population increase vs. population size
2. Adjust the appearance of the plot:
   - Add a plot title and axis labels
   - Change line thickness and appearance
   - Experiment with other changes
3. Experiment with model parameters:
   - Try to increase the initial population over the capacity of the environment
4. Model the population* of the USA and compare it with real data.

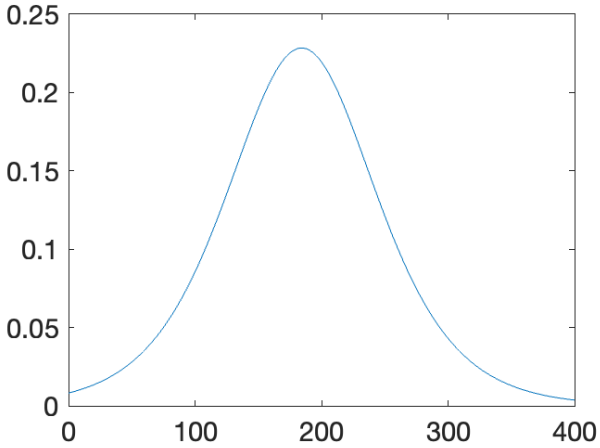
** For the population model of the USA, set the step size to 1 month, the initial population size to 5.2 million in 1790, the growth rate to 0.0237, and the capacity of the environment to 460 million.*

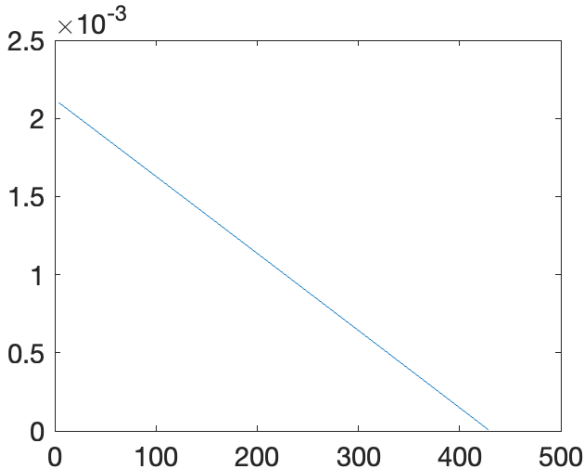
Solution 3.1: Logistic growth model

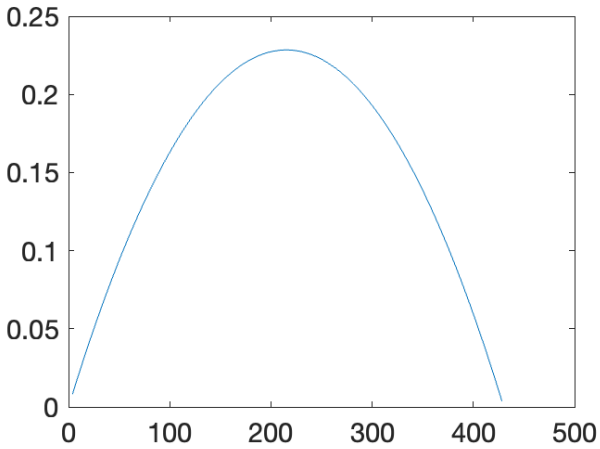
```python
import numpy as np
import matplotlib.pyplot as plt

# Parameters and Time
P0, r, K = 3.9, 0.0255, 430
t = np.arange(0, 400, 1/12)

# Vectorized Logistic Growth (Analytical Solution)
# P(t) = K / (1 + ((K - P0) / P0) * exp(-rt))
P = K / (1 + ((K - P0) / P0) * np.exp(-r * t))
P_diff = r * P * (1 - P / K) * (1/12)

# Plotting
fig, axs = plt.subplots(2, 2, figsize=(10, 8))
plots = [
    (t, P, "Population vs Time"),
    (t, P_diff, "Growth Amount vs Time"),
    (P, P_diff / P, "Per Capita Growth vs Pop"),
    (P, P_diff, "Growth Amount vs Pop")
]

for ax, (x, y, title) in zip(axs.flat, plots):
    ax.plot(x, y)
    ax.set_title(title)

```

Solution 3.1: Logistic growth model





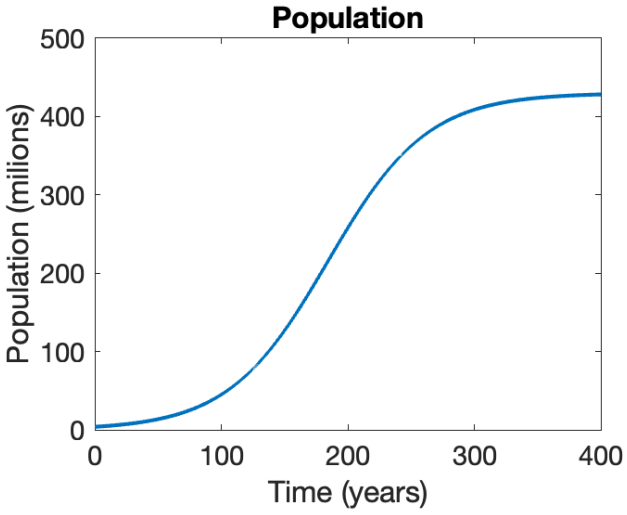


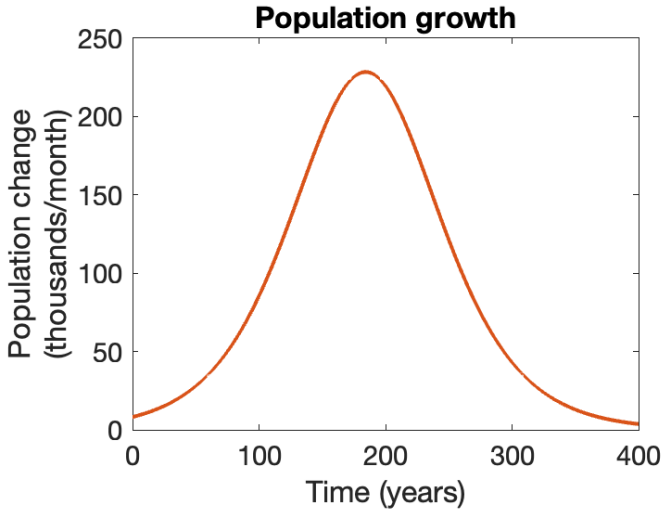


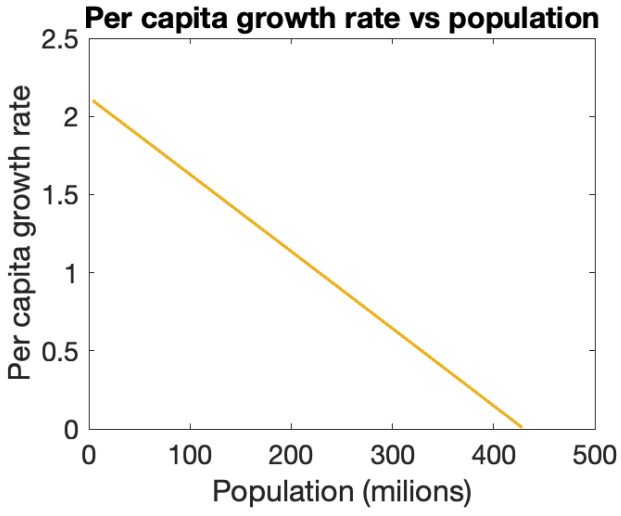
Solution 3.2: Adjusted plot

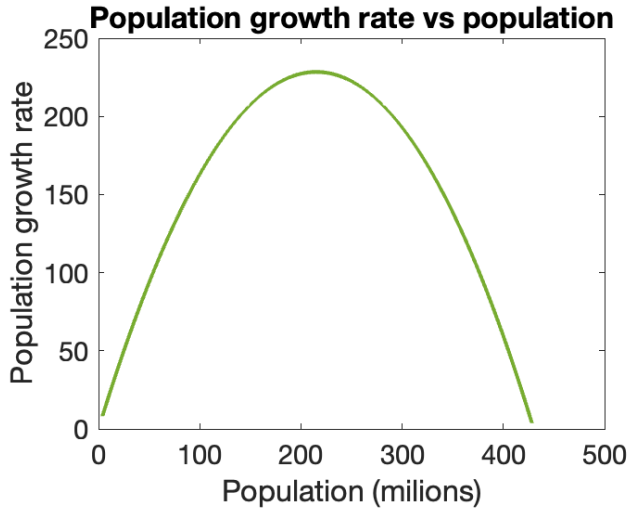
```
import numpy as np
import matplotlib.pyplot as plt

# Params & Time
P0, r, K, dt = 3.9, 0.0255, 430, 1/12
t = np.arange(0, 400 + dt, dt)
P = np.zeros(len(t))
P[0] = P0

for i in range(len(t) - 1):
    P[i+1] = P[i] + r * P[i] * (1 - P[i] / K) * dt

# Derived Data
P_diff = np.concatenate([[np.nan], np.diff(P)])
plots = [(t, P, "Pop vs Time"), (t, P_diff, "Growth vs
    Time"),  (P, P_diff/P, "Per Capita vs Pop"), (P,
    P_diff, "Growth vs Pop")]

fig, axs = plt.subplots(2, 2, figsize=(10, 7))
for ax, (x, y, title) in zip(axs.flat, plots):
    ax.plot(x, y)
    ax.set_title(title)

plt.tight_layout()
plt.show()
```

Solution 3.2: Adjusted plot





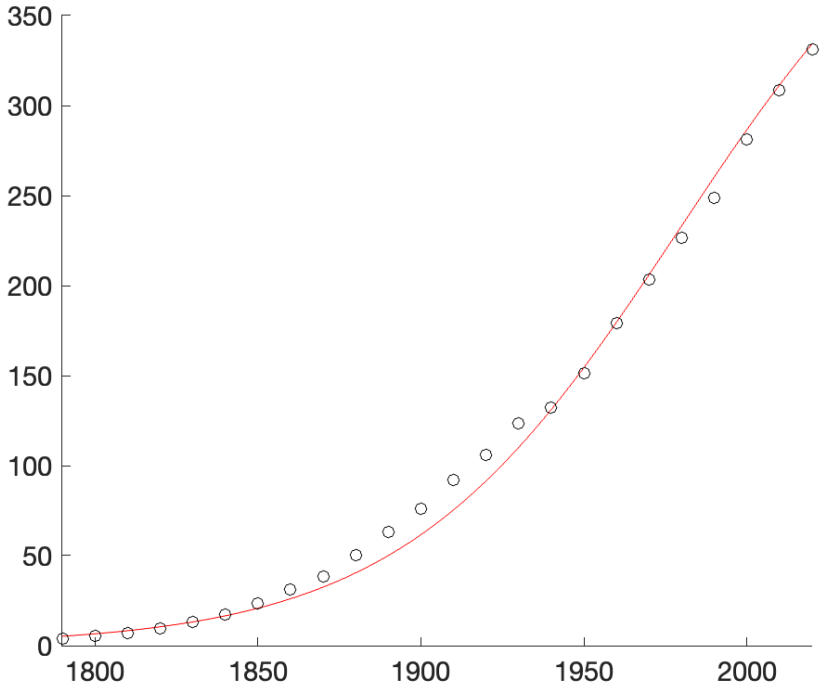




Solution 3.3: US population

```python
import numpy as np
import matplotlib.pyplot as plt

# Parameters
r, P0, K, start_year = 0.0237, 5.2, 460, 1790
t_usa = np.arange(0, 240, 10)
P_usa = np.array([3.9, 5.3, 7.2, 9.6, 12.9, 17.1, 23.2,
    31.4, 38.6, 50.2, 63.0, 76.2, 92.2, 106.0, 123.2,
    132.2, 151.3, 179.3, 203.3, 226.5, 248.7, 281.4, 308.7,
    331.4])

dt = 1/12
t_num = np.arange(0, 230 + dt, dt)
P_num = np.zeros(len(t_num))
P_num[0] = P0

for i in range(len(t_num) - 1):
    P_num[i+1] = P_num[i] + r * P_num[i] * (1 - P_num[i] /
        K) * dt

# Plotting
plt.plot(t_num + start_year, P_num, 'r-', label='Logistic
    Model')
plt.plot(t_usa + start_year, P_usa, 'ko', label='Real Data')
plt.axis([1790, 2020, 0, 350])
```

Solution 3.3: US population



Section 4: Leslie Matrix Model

Limitations of previous models

- So far, our models (Malthusian, Logistic) assumed all individuals in a population are identical.
- **In reality, age matters:**
  - **Reproduction:** Young individuals (larvae, babies) cannot reproduce yet.
  - **Survival:** Survival rates often depend on age (e.g., higher mortality in very young or very old individuals).
- To capture this, we need an **age-structured model**.
- The most common discrete-time model for this is the **Leslie Matrix**.

The Leslie Matrix Model

Developed by Patrick H. Leslie in 1945.

- It tracks the number of females in different age classes at discrete time steps.
- **Variables:**
  - $n_i(t)$: Number of individuals in age class i at time t.
  - F_i: **Fecundity** (fertility) rate. Average number of female offspring produced by a female in age class i.
  - S_i: **Survival** rate. Probability that a female in age class i survives to age class $i + 1$.

Mathematical Formulation

If we have 3 age classes, the population vector at time $t+1$ is calculated as:

$$n_1(t+1) = F_1 n_1(t) + F_2 n_2(t) + F_3 n_3(t) \quad \textbf{New offspring} \text{ (born from all classes)}$$
$$n_2(t+1) = S_1 n_1(t) \quad \textbf{Survivors} \text{ from class 1 moving to 2}$$
$$n_3(t+1) = S_2 n_2(t) \quad \textbf{Survivors} \text{ from class 2 moving to 3}$$

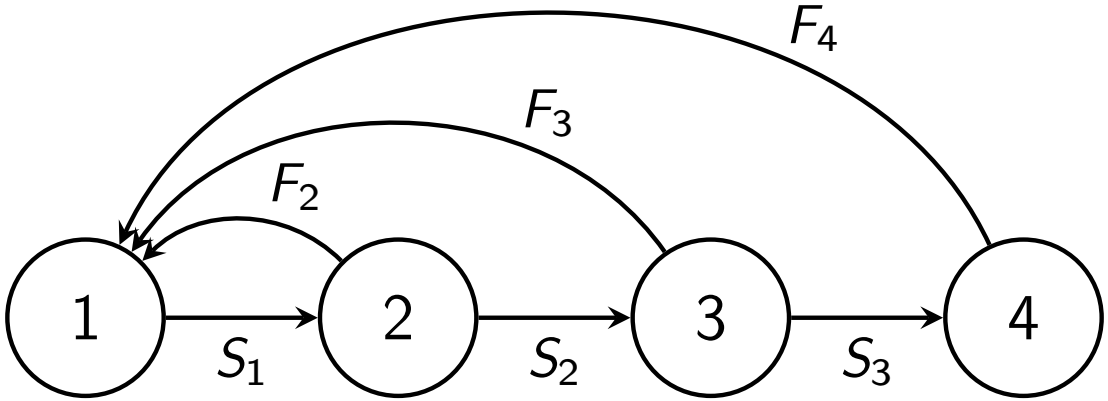
This can be written in matrix form:

$$\begin{bmatrix} n_1 \\ n_2 \\ n_3 \end{bmatrix}_{t+1} = \begin{bmatrix} F_1 & F_2 & F_3 \\ S_1 & 0 & 0 \\ 0 & S_2 & 0 \end{bmatrix} \begin{bmatrix} n_1 \\ n_2 \\ n_3 \end{bmatrix}_t$$

$$\mathrm{n}_{t+1} = \mathrm{L} \cdot \mathrm{n}_t$$

Life Cycle with Age-Class Structure

Individuals are grouped into discrete age classes of equal duration.



- **Circles:** Represent specific age classes or ages (1 to 4).
- **Horizontal Arrows (S_x):** Represent **survival probabilities**. Probability that an individual of age x survives to age $x + 1$. There is no survival beyonf group 4.
- **Curved Arrows (F_x):** Represent **births (fecundity)**. Arrows lead back to Age 1 because newborns enter the first class. Arrows emerging from ages 2, 3, and 4 indicate these ages are reproductive.

Matrix Representation of the Life Cycle

The life cycle diagram translates directly into a **projection matrix** (Leslie Matrix).

The Equation:

$$\mathrm{n}_{t+1} = \mathrm{L} \cdot \mathrm{n}_t$$

The Matrix Expansion:

$$\begin{bmatrix} n_1 \\ n_2 \\ n_3 \\ n_4 \end{bmatrix}_{t+1} = \begin{bmatrix} 0 & F_2 & F_3 & F_4 \\ S_1 & 0 & 0 & 0 \\ 0 & S_2 & 0 & 0 \\ 0 & 0 & S_3 & 0 \end{bmatrix} \begin{bmatrix} n_1 \\ n_2 \\ n_3 \\ n_4 \end{bmatrix}_t$$

Structure of the Leslie Matrix

$$L = \begin{bmatrix} F_1 & F_2 & F_3 & \dots & F_k \\ S_1 & 0 & 0 & \dots & 0 \\ 0 & S_2 & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \dots & \vdots \\ 0 & 0 & \dots & S_{k-1} & 0 \end{bmatrix}$$

- **Top Row:** Fertility rates (production of new offspring).
- **Sub-diagonal:** Survival rates (transition to next age group).
- **All other entries:** Zero (you cannot skip an age or get younger).

Long-term Dynamics

What happens if we simulate this for many time steps?

- **Population Growth:** The total population eventually grows (or declines) at a constant rate λ.
- **Stable Age Distribution:** The *proportion* of individuals in each age class becomes constant.
  - Example: 60% juveniles, 30% adults, 10% seniors.
  - This happens regardless of the initial population state!

Note for linear algebra enthusiasts: λ is the dominant eigenvalue of L, and the stable age distribution is the corresponding eigenvector.

Task 4.1: Defining the Model

Scenario: A population of hypothetical birds

- **Class 1 (Chicks, 0-1 yr):** Do not reproduce ($F_1 = 0$). 60% survive to next year ($S_1 = 0.6$).
- **Class 2 (Young Adults, 1-2 yr):** Moderate reproduction ($F_2 = 1.5$). 50% survive ($S_2 = 0.5$).
- **Class 3 (Older Adults, 2-3 yr):** High reproduction ($F_3 = 2.0$). They die after this year ($S_3 = 0$).

Your Task:

1. Construct the Leslie Matrix L in Python (using NumPy).
2. Define an initial population vector n_0: 100 chicks, 0 adults.

Solution 4.1: Defining the Model

```
1  import numpy as np
2
3  # Define parameters
4  F1 = 0;    F2 = 1.5; F3 = 2.0
5  S1 = 0.6; S2 = 0.5
6
7  # Construct Leslie Matrix
8  L = np.array([[F1, F2, F3],
9                [S1, 0,  0],
10               [0,  S2, 0]])
11
12 # Initial population (100 chicks, 0 young adults, 0 old
       adults)
13 n0 = np.array([100, 0, 0])
```

Task 4.2: Simulation Loop

Your Task:

- Simulate the population for 20 years.
- Store the population vector for each year.
- *Hint: You will need a matrix to store results, where columns represent time steps.*

Solution 4.2: Simulation Loop

```python
years = 20
# Pre-allocate matrix (3 rows, years+1 columns)
pop_history = np.zeros((3, years + 1))

# Set initial year (Python is 0-indexed)
pop_history[:, 0] = n0

# Loop through time
for t in range(years):
    # n(t+1) = L @ n(t)
    # The @ operator performs matrix multiplication
    pop_history[:, t+1] = L @ pop_history[:, t]
```

Task 4.3: Analyzing Results

Your Task:

1. Calculate the **total population** for each year (sum of all age classes).
2. Plot the Total Population over time.
3. Plot the specific age classes over time.

Solution 4.3: Plotting

```
import matplotlib.pyplot as plt

# Calculate total population (sum each column, axis=0)
total_pop = np.sum(pop_history, axis=0)
t = np.arange(years + 1)

# Plot
plt.figure(figsize=(10, 6))

plt.subplot(2,1,1)
plt.plot(t, total_pop, 'k-', linewidth=2)
plt.title('Total Population Growth'); plt.grid(True)

plt.subplot(2,1,2)
plt.plot(t, pop_history[0, :], 'b-o', label='Chicks')
plt.plot(t, pop_history[1, :], 'r-o', label='Young')
plt.plot(t, pop_history[2, :], 'g-o', label='Old')
plt.legend(); plt.grid(True)
plt.show()
```

Task 4.4: Stable Age Distribution (Advanced)

We claimed earlier that the *proportion* of age groups becomes constant. **Your Task:**

- Normalize the population data at each step (divide each column by the total population of that column).
- Inspect the last few columns of this normalized matrix.
- Are the proportions changing?

Solution 4.4: Stable Age Distribution

```python
# Normalize to find proportions
# axis=0 means we sum down the columns
proportions = pop_history / np.sum(pop_history, axis=0)

# Display the proportions for the last year (-1 index)
print("Final Age Distribution:")
print(proportions[:, -1])

# Check the previous year to see if it stabilized
print("Previous Year Distribution:")
print(proportions[:, -2])
```

You should see that the vector values stabilize, confirming the Stable Age Distribution theorem.